Master's Thesis

Vehicle Speed Estimation Based on Artificial Intelligence Algorithms

Author:
Andrzej Skrodzki

Budapest University of Technology and Economics
Department of Control for Transportation and Vehicle Systems

Supervisor:
Donát Rácz

Master's Thesis,
December 10, 2025

Contents

Chapter 1

Introduction

Accurate vehicle speed estimation plays a critical role in modern automotive systems, contributing to various functionalities such as tire slip calculation, advanced driver-assistance systems (ADAS), intelligent transportation systems (ITS), and autonomous driving technologies. Conventionally, the measurement of vehicle speed has been accomplished through sensors that measure the motion of the vehicle. These common methods mainly measure the position, velocity or acceleration of the vehicle via wheel speed sensors, inertial measurement unit (IMU) or the data received from the GNNS signals. However, it must be acknowledged that these methodologies are not without their limitations. The measurements ontained from any sensors are affected by noise, biases and potential failures that lead to inaccuracies and bad precision in the estimated vehicle speed.

The measurements obtained from the sensors are processed through various estimation algorithms to derive the vehicle speed. Traditional non-linear estimation techniques include simple averaging methods, model-based estimators, and Kalman filtering approaches. While these methods have been proven effective in many scenarios, they often rely on predefined models and assumptions that may not fully capture the complex dynamics of vehicle motion, especially under varying road conditions and driving behaviors.

As the sequential data received from the communication bus (CAN) of the vehicle, the approach of using artificial intelligence (AI) has gained interest in recent years. These methods allow to describe the highly non-linear relationships between the input sensor signals and the vehicle speed without the need for explicit modeling. Various AI architectures have been explored for their ability to learn temporal dependencies and patterns in sequential data.

This thesis explores the development and validation of AI-based models for real-time vehicle speed estimation using CAN signal data from a passenger car. The estimation performance is evaluated against GPS measurements, which serve as ground truth. Moreover, the study explores hybrid approaches that integrate artificial intelligence (AI) with conventional filtering methods, with the objective of combining the strengths of both approaches.

Chapter 2

Vehicle speed

2.1 Defining Vehicle Speed

The speed of an arbitrary point in space is defined as the magnitude of the velocity vector of that point, that is considered in the inertial reference frame ($\mathcal{R}_{[x_0,y_0,z_0]}$, later denoted as $\mathcal{R}_0$ for simplicity). The velocity vector can be written as the time derivative of the position vector, where the position vector points from the origin of the inertial reference frame to the chosen moving point in space. However, when considering a moving vehicle in the same space, a relatively constrained set of points, i.e. a body, has to be considered instead of one moving point. In order to use the previously mentioned formula for determining the speed of the vehicle, one point has to be chosen on the vehicle that will represent the whole. This will be the center of gravity (CoG) that is the average location of the weight of the vehicle. With this in mind the vehicle speed can be determined as follows:

$$v_{CoG} = \|[\mathbf{v}_{CoG}]_{\mathcal{R}_0}\| = \left\| \frac{d[\mathbf{r}_{CoG}]_{\mathcal{R}_0}}{dt} \right\|, \tag{2.1}$$

where v_{CoG} is the vehicle speed, $[\mathbf{v}_{CoG}]_{\mathcal{R}_v}$ is the velocity vector of the CoG in the inertial reference frame and $[\mathbf{r}_{CoG}]_{\mathcal{R}_0}$ is the position vector of the CoG in the inertial reference frame. This way the velocity and the position vectors can be expressed in a 3 dimensional space as:

- position vector: $[\mathbf{r}_{CoG}]_{\mathcal{R}_0} = \begin{bmatrix} r_x \\ r_y \\ r_z \end{bmatrix}_{\mathcal{R}_0}$
- velocity vector: $[\mathbf{v}_{CoG}]_{\mathcal{R}_0} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}_{\mathcal{R}_0}$

As the moving vehicle in space is considered as a 3 dimensional body, the orientation of

it can be described with a body-fixed reference frame denoted by $\mathcal{R}_{[x_v,y_v,z_v]}$ (later denoted by $\mathcal{R}_v$ for simplicity), where x_v points forward along the longitudinal axis of the vehicle frame, y_v points sideways to the left along the lateral axis and z_v points upwards to complete the right-handed coordinate system. Let this coordinate system be placed in the CoG of the vehicle. In order to describe the velocity vector in this body-fixed reference frame, the angles between the two reference frames have to be defined. These angles are the following:

- angle of rotation around axis x_v: δ_{x_v},
- angle of rotation around axis y_v: δ_{y_v},
- angle of rotation around axis z_v: δ_{z_v}.

With the help of these three angles, the velocity vector can be expressed in the body-fixed reference frame with the help of the transformation matrix $T_{\mathcal{R}_0 \to \mathcal{R}_v}$ between the two reference frames. From this the transformation matrix is as follows:

$$T_{\mathcal{R}_0 \to \mathcal{R}_v} = R_z(\delta_{x_v}) \cdot R_y(\delta_{y_v}) \cdot R_x(\delta_{z_v}), \tag{2.2}$$

where R_x, R_y and R_z are the rotation matrices around the respective axes.

It is important to note that throughout this thesis, the vehicle is assumed to be moving on a flat surface. This means that the angles of rotation around the x_v and y_v axes are considered negligible, i.e., $\delta_{x_v} \approx 0$ and $\delta_{y_v} \approx 0$. Consequently, the transformation matrix simplifies to a rotation around the z_v axis only:

$$T_{\mathcal{R}_0 \to \mathcal{R}_v} = R_z(\delta_{z_v}). \tag{2.3}$$

This simplified 2D representation of the vehicle speed in the body-fixed reference frame is illustrated in Figure 2.1.

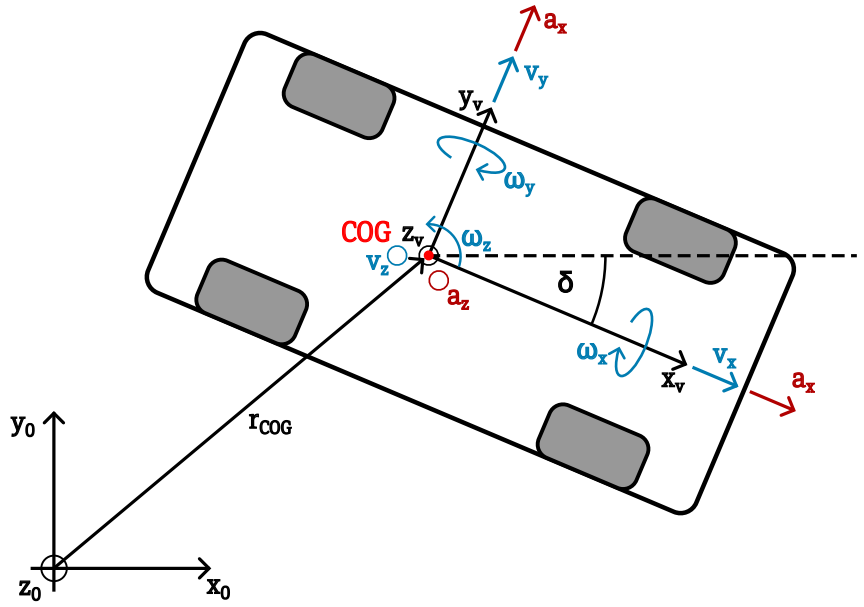


Figure 2.1: Vehicle speed in body-fixed reference frame in 2D space

With the help of the transformation matrix the velocity vector in the body-fixed reference frame can be determined as:

$$[\mathbf{v}_{CoG}]_{\mathcal{R}_v} = T_{\mathcal{R}_0 \to \mathcal{R}_v} \cdot [\mathbf{v}_{CoG}]_{\mathcal{R}_0} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}_{\mathcal{R}_v}, \tag{2.4}$$

where the longitudinal speed v_x is the component of the velocity vector along the x_v axis, the lateral speed v_y is the component of the velocity vector along the y_v axis and the vertical speed v_z is the component of the velocity vector along the z_v axis.

2.2 Measurements for determining vehicle speed

There are various ways to determine the speed of the vehicle. In this section the focus is on exploring these different ways and analyzing them.

2.2.1 Engine speed

The first way is calculating the wheel and vehicle speed from the rotational speed of the engine [1]. It is a rather theoretical approach but it gives an example of a simplified overview how an engine is connected to the wheels through the gearbox and the differential. The setup can be seen on Figure 2.2.

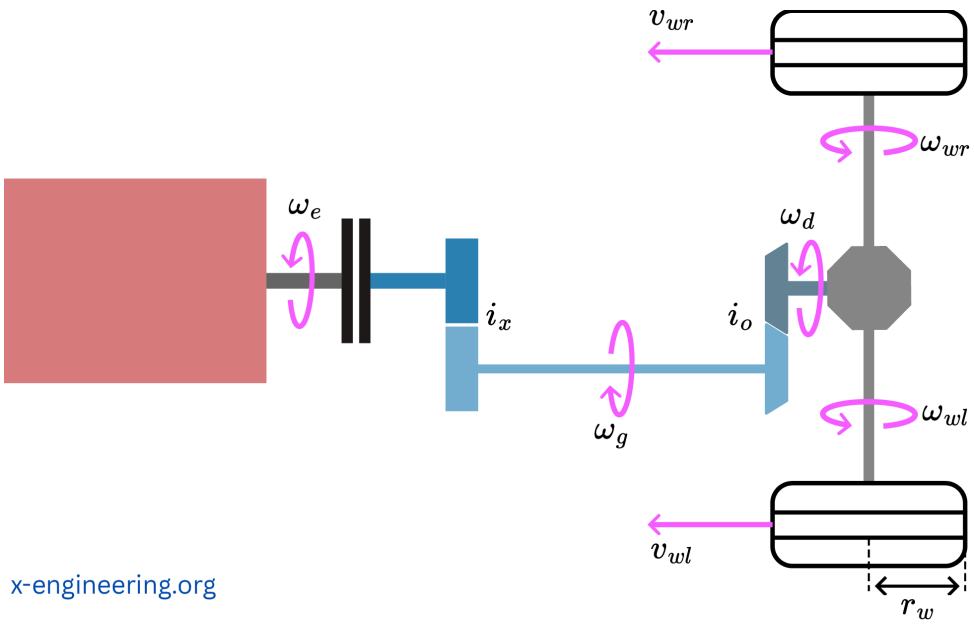


Figure 2.2: Calculating vehicle speed from engine speed [1]

The formula for determining the vehicle speed is the following:

$$v_v = \frac{N_e \cdot \pi \cdot r_\omega}{30 \cdot i_x \cdot i_0} \left[\frac{m}{s}\right], \tag{2.5}$$

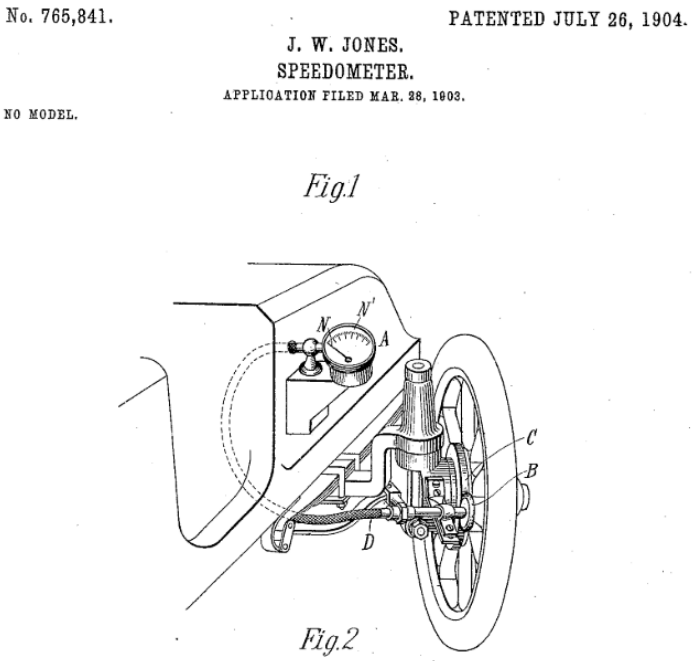
where N_e [rpm] is the engine speed, r_ω [m] is the wheel radius, i_x [-] is the gear ratio of the engaged gear and i_0 [-] is the gear ratio of the differential.

The main advantage of this method comes from its simplicity. Based on a one measured data, that is the engine shaft's rotational speed (N_e), the longitudinal vehicle speed can be determined. Contrary to this it can only give an estimate due to the simplifications it considers. It doesn't take in account the slip in the clutch between the gearbox and the engine shaft, the longitudinal and lateral slip of the tires, the difference in speed of the left and right wheels in case of traveling in a curve, the case when the gearbox and the engine are detached while the vehicle is being in neutral (gear 0) and when the wheels are locked but the vehicle is still motion. Considering the points mentioned above, this method could be used in highway driving where the vehicle travels with constant high speed on a straight path with little curvature.

2.2.2 Wheel speed

Following the previous method where the measured unit was the rotational speed of the engine's shaft, here the shift goes to measuring the rotational speed of the vehicle's wheel. This is done by a speedometer. The first speedometer is credited to Josip Belušić [2]. His invention in 1888 was based on the electromagnetic induction which worked the following way. A flexible rotating cable is connected to the vehicle wheel's rotating

shaft via a friction disk or a pair of gears that translates the rotational motion to a disk shaped magnet. This rotating magnet creates a fluctuating magnetic field which induces a rotating movement in the speed cup that surrounds the magnet. However the speed cup's movement is limited by a spring. This spring will allow greater rotation when greater torque applies to the speed cup that is in the case of faster rotation from the magnet. The needle that showed the speed of the vehicle is linked to the speed cup. This way the needle moved proportionally to the speed of the vehicle [3]. Later this principle was patented by Otto Schulze in 1903 [4] and Joseph W Jones in 1904 [5]. Both of the patents share the same principle.


No. 765,841. PATENTED JULY 26, 1904.

J. W. JONES.

SPEEDOMETER.

APPLICATION FILED MAR. 28, 1903.

NO MODEL.

Fig.1

Fig.2

Figure 2.3: Speedometer patented by Joseph W Jones in 1904 [5]

The rather mechanical method described above was in use for several decades but it has multiple downsides: wear of the elements due to the mechanical connections, uncertainty coming from the mechanical measurement method and the fact of having the speed from only one wheel. To compensate the points mentioned before, electric wheel speed sensors were introduced to enhance the precision of the measurements. There are two types of electronic wheel speed sensors: passive and active.

The principle of the passive wheel speed sensor is the following. A toothed reluctant ring is mounted on the rotating wheel. Above this ring a variable reluctance (VR) sensor is placed. When the wheel starts to rotate each tooth changes the magnetic flux through the coil of the sensor which induces an alternating voltage signal. The frequency of this analog sine wave is proportional to the rotational speed. However, the active wheel speed sensor uses a semiconductor magnetic sensor (Hall or magneto-resistive sensor) and a magnetic encoder ring attached to the rotating wheel. As the ring rotates, the magnetic

field changes are detected by the magnetic sensor. These changes are then converted to square wave digital signals [6]. The active sensors can overall provide a more precise measurement than the passive sensors.

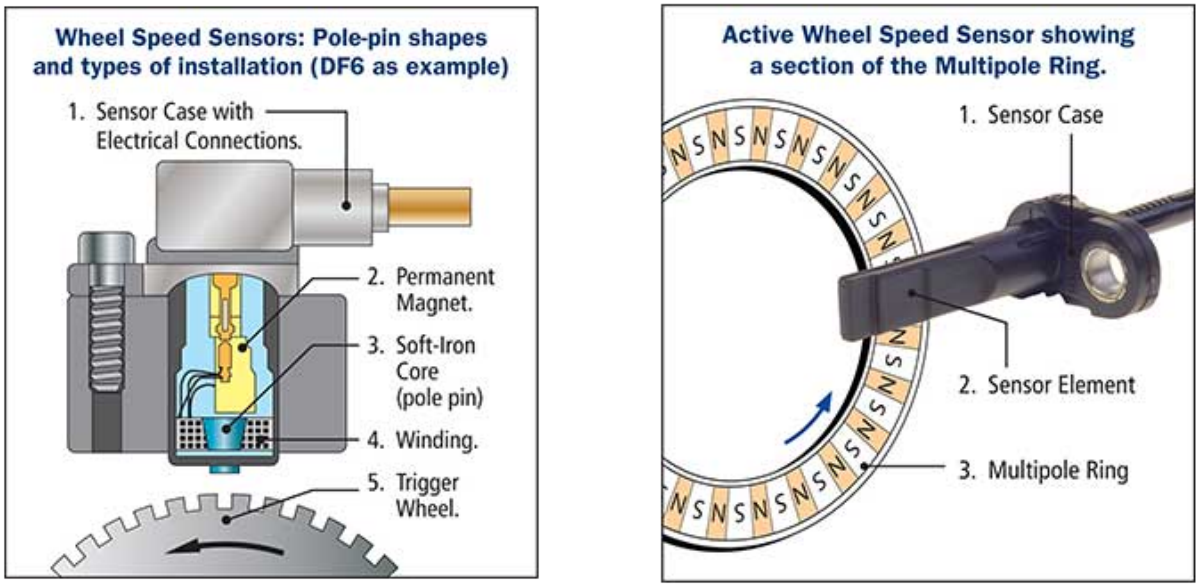


Figure 2.4: Passive (left) and active (right) wheel speed sensors from [6]

In the case of these wheel speed sensors, the output is a rotational speed. From this rotational speed the ground speed of the specific wheel can be determined with the help of the diameter of the wheel. This measurement can be done for all the wheels, which will serve as an input for later discussed estimator algorithms.

2.2.3 Inertial Measurement Unit

The inertial measurement unit (IMU) can also provide relevant measurement data to determine the vehicle speed. An IMU consist of a set of accelerometers and gyroscopes to determine the acceleration along and the rotational rate around the longitudinal, lateral and vertical axes. This gives two vectors: acceleration $\mathbf{a}$ and angular velocity vector $\dot{\boldsymbol{\eta}}$ in the body fixes coordinate system:

$$\mathbf{a} = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} \quad \text{and} \quad \dot{\boldsymbol{\eta}} = \begin{bmatrix} \dot{\varphi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} \tag{2.6}$$

Before calculating the velocity, the gravitational vector has to be deducted from the measured acceleration vector that can be done with the help of the angular velocity vector. From here, as the velocity vector's time derivative equals the acceleration vector,

the velocity can be derived by an integral:

$$\mathbf{v}(t) = \mathbf{v}(t_0) + \int_{t_0}^{t} \mathbf{a}(\tau) d(\tau) \tag{2.7}$$

It is important to mention here that this measurement method is not robust by itself. Apart from setting the initial velocity of the vehicle, the biases in the accelerometers lead to integration drift that accumulates into large errors over time. To mitigate this issue this method has to be combined with other velocity measurements and estimators.

2.2.4 Global Navigation Satellite System

Global Navigation Satellite System (GNSS) refers to any constellation that provides global positioning, navigation and timing services. The currently available GNSS systems in Europe are GPS, Galileo, Glonass and BeiDou [7]. The velocity of the vehicle can be obtained by exploiting the raw Doppler, the carrier phase or the pseudorange measurements with a GNSS receiver.

The raw Doppler measurement utilizes the Doppler shift between the transmitted and received signal frequencies. The Doppler shift can be expressed as:

$$D = \Delta f = f_0 - f_r, \tag{2.8}$$

where f_0 is the transmitted frequency and f_r is the received frequency.From this the radial velocity between the satellite and the receiver can be determined as:

$$\dot{\rho} = v_r \approx -\frac{c}{f_0} \cdot D, \tag{2.9}$$

where c is the speed of light in vacuum. With the help of the radial velocities from multiple satellites, the velocity vector of the receiver can be determined in the Earth-Centered Earth-Fixed (ECEF) coordinate system (figure 2.5).

Carrier phase measurement uses the phase difference between the transmitted and the received carrier signals to determine the radial velocity. This method provides higher accuracy compared to the Doppler method but requires resolving integer ambiguities associated with the carrier phase measurements. The range rate can expressed from:

$$\frac{\Delta(\Phi\lambda)}{\Delta t} \approx \dot{\rho} + c(\dot{\delta}t_r - \dot{\delta}t_s) + \text{small residual terms}, \tag{2.10}$$

where Φ is the carrier phase measurement in cycles, λ is the wavelength of the carrier signal, $\dot{\delta}t_r$ and $\dot{\delta}t_s$ are the receiver and satellite clock drifts respectively. With this the

radial velocity can be approximated in discrete time as:

$$\dot{\rho} \approx \frac{\Phi(t_{k+1})\lambda - \Phi(t_k)\lambda}{t_{k+1} - t_k} - c(\dot{\delta t}_r - \dot{\delta t}_s). \tag{2.11}$$

Pseudorange differentiation method determines the radial velocity by differentiating the pseudorange measurements over time. The pseudorange is the estimated distance between the receiver and the satellite. This range can be expressed as:

$$P = \rho + c(\delta t_r - \delta t_s) + \text{other error terms}, \tag{2.12}$$

where P is the pseudorange, ρ is the true range, δt_r and δt_s are the receiver and satellite clock biases respectively. By differentiating the pseudorange over time, the radial velocity can be approximated as:

$$\frac{dP}{dt} \approx \dot{\rho} + c(\dot{\delta t}_r - \dot{\delta t}_s) + \text{small residual terms}. \tag{2.13}$$

The radial velocity can be approximated in discrete time as:

$$\dot{\rho} \approx \frac{P(t_{k+1}) - P(t_k)}{t_{k+1} - t_k} - c(\dot{\delta t}_r - \dot{\delta t}_s). \tag{2.14}$$

Figure 2.5: Trilateration (left) from [8] and ECEF coordinate system (right) from [9]

2.3 Estimating vehicle speed

2.3.1 Accelerometer and speedometer

The simplest method to estimate the vehicle speed is to take the measurements from the speedometers at all the wheels. The longitudinal speed of each wheel can be determined from the rotational speed data from the sensors. These values then can be transformed into the CoG of the vehicle. There the transformed longitudinal velocities can be averaged

to give an estimate of the vehicle speed at the CoG:

$$v_{est,CoG} = \frac{v_{cog}^{FR} + v_{cog}^{FL} + v_{cog}^{RR} + v_{cog}^{RL}}{4}, \tag{2.15}$$

where v_{cog}^{FR}, v_{cog}^{FL}, v_{cog}^{RR} and v_{cog}^{RL} are the longitudinal speeds of the front right, front left, rear right and rear left wheels transformed to the CoG of the vehicle respectively.

Another simple algorithm for vehicle speed estimation is presented in [10], where the vehicle speed is determined by the measurements from the IMU's longitudinal accelerometer and the wheel speed sensors. The formula that estimates the speed is described as follows:

$$v_{est,CoG}(k) = v_{simple}(k_{nobrake}) + \sum_{i=k_{nobrake}}^{k} a_{meas}(i) \cdot dt, \tag{2.16}$$

where the k is the time step counter, $k_{nobrake}$ is the last time index where there was no braking, a_{meas} is the acceleration value form the IMU's longitudinal accelerator and dt is the sampling time. This method works in a way that estimation is taken apart into two categories based on whether the vehicle is barking or not. The simplicity comes at the price of inaccuracies. The estimator does not perform well when accelerometer offset is set and the dynamic change of the wheel radius is not implemented. An other difficulty comes from dealing with the noisy measurements from the wheel speed sensors and the IMU. A similar algorithm to this one will be presented later in section 5.1. The results of the simple algorithm can be seen on figure 2.6.

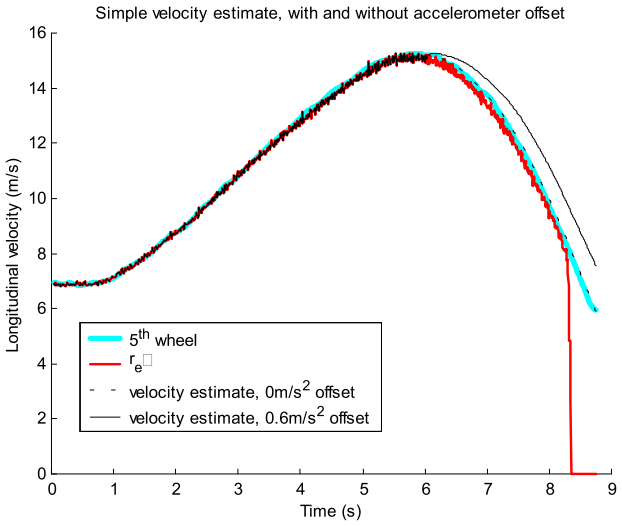


Figure 2.6: Simple algorithm for estimating vehicle speed from [10]

2.3.2 Kalman filter

To elevate the performance of the previously described methods, a Kalman filter can be added to the system. At a high level the Kalman filter estimates the vehicle speed from noisy sensor data by iterating between the following two steps:

1. Prediction
    - The filter takes the last estimate and predicts the next state's longitudinal or lateral velocity based on the simple dynamic motion model.
    - The filter provides an estimate and an uncertainty measure for the prediction.
2. Correction
    - Based on the predicted data and the new measurement data the filter calculates an optimal blend of the two values.
    - A posterior estimate is determined that is more accurate and less noisy than the pure measurement data.

Following this structure a Kalman filter is proposed in [11] to estimate the vehicle's speed. To enhance the accuracy of the estimator, 4 different driving situations are described with 4 different covariance matrices.

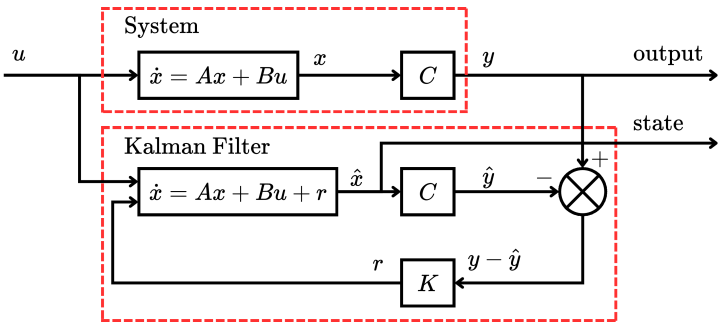


Figure 2.7: Vehicle speed estimation with Kalman filter from [11]

To elevate the usage of Kalman filter a more sophisticated and non-linear vehicle model can be used to predict the vehicle speed. In [12] an Extended Kalman filter (EKF) is introduced with vehicle model that runs parallel with the system itself. The overview of the filtering approach can be seen on figure 2.8. An EKF based estimator will be discussed later in section 5.3.

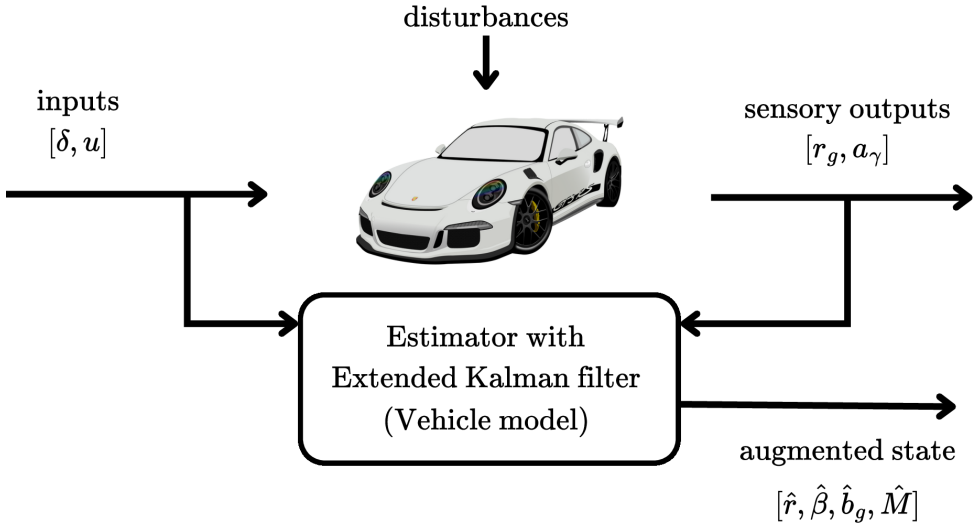


Figure 2.8: Extended Kalman filter for vehicle speed estimation from [12]

2.3.3 Fuzzy logic

An other approach to estimate the vehicle speed is using fuzzy logic. Fuzzy logic offers heuristic approach to handle uncertainty and imprecision of the sensors. In [11] a comprehensive method is described using the Fuzzy logic to determine the vehicle speed using different driving scenarios.

The main motive behind the proposed estimator is determining the weights of the sensor signal values, that are the wheel speeds and the longitudinal acceleration. All these values coming from the senors receive a weighting factor k_i, where $i \in [1...5]$. The weighted average is then formulated as follows:

$$v_{est,CoG}(t) = \frac{\sum_{i=4}^{4} k_i \cdot v_{Ri,C}(t) + k_5[v_{est,CoG}(t-1) + T_S \cdot a_{X,C}(t)]}{\sum_{i=1}^{5} k_i}, \tag{2.17}$$

where the weights $k_1...k_4$ are for the wheel speed sensors and the weight k_5 is for the acceleration and all of them are determined with the rules of the fuzzy logic. The weights are determined based on the driving situation that are strong acceleration, acceleration, rolling, braking, strong braking. Based on these situations a set of weights are defined in advance. With this approach the estimator can adapt to different driving situations and provide a more accurate vehicle speed estimation. The structure of the fuzzy logic system can be seen on figure 2.9.

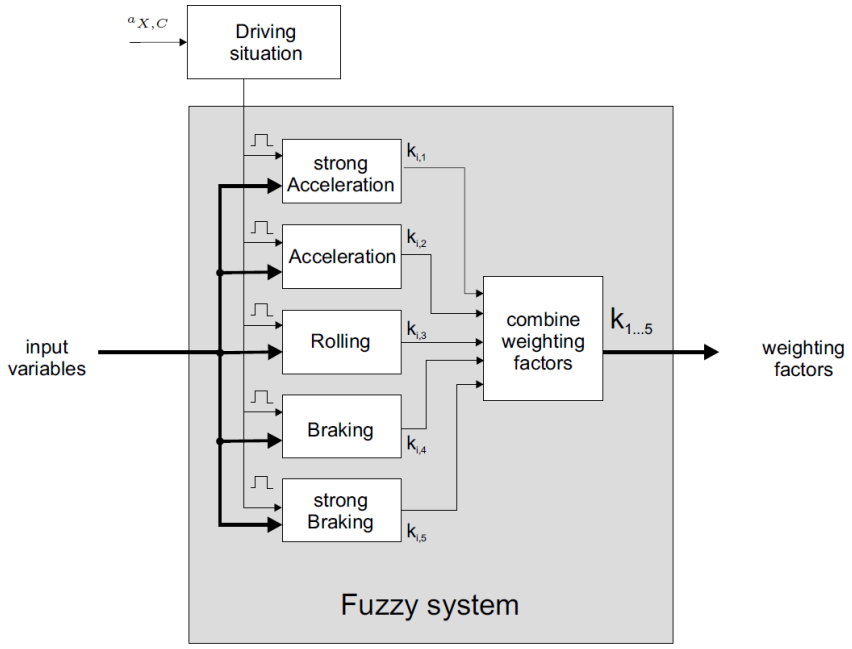


Figure 2.9: Vehicle speed estimation with Fuzzy logic from [11]

2.3.4 GNSS

There are multiple algorithms to determine the velocity of the vehicle using the GNSS system. In article [13] 4 algorithms are described and compared. These methods are the following: Raw Doppler (RD), Time-Differenced Pseudorange (TDPR), Time-Differenced Carrier Phase (TDCP) and Double-Differenced Carrier Phase (DDCP). These algorithms were both tested in static and dynamic environments and postprocessed after the tests.

The most accurate algorithm in dynamic environments is the RD method that utilizes the Doppler shift from the carrier phase difference of the transmitted and received signal. The measurement model can be written as:

$$\lambda D = \dot{\rho} + c(d\dot{t}_R - d\dot{t}_S) + \dot{I}_\rho + \dot{T}_\rho + \dot{\epsilon}_\rho, \tag{2.18}$$

where λ is the wavelength, D is the Doppler shift, $\dot{\rho}$ is the rate of the pseudorange, c is the speed of light in vacuum, $d\dot{t}_R$ and $d\dot{t}_S$ are the satellite and receiver clock drifts, $\dot{I}_\rho$ and $\dot{T}_\rho$ are the rates of change in the ionospheric and tropospheric delays and $\dot{\epsilon}_\rho$ combines the unmodeled error and observation noise. From this model the velocity of the vehicle can be determined in the global ECEF frame. The tests with this method resulted a [cm/s] level accuracy that is precise and accurate enough for driver assistance features.

The significant advantage of this algorithm is definitely the precision that it provides and can be used to enhance the previously described estimators. The disadvantages are that the GNSS receivers are not standard in passenger and commercial vehicles leaving little space for wide-spread usage of the advantages, the 10 [Hz] sampling time is relatively

low that requires some type of interpolation between the samples and that the determined velocity is not in the vehicle's body fixed frame. The last disadvantage comes from the situations when the signals from the satellites are not visible in case of driving in a tunnel or urban areas with tall buildings.

2.3.5 Artificial Intelligence

After discussing the more conventional estimators that rely on deterministic formulas for calculating the vehicle speed, let the focus be shifted to the main topic of this thesis: Artificial Intelligence (AI) for vehicle speed estimation.

Artificial intelligence has been a greatly investigated field in the recent years for a wide range of applications. Despite the robustness and highly safety critical approach of the vehicle industry, the investigation of possible applications has been started. There are two main approaches:

1. Integrating AI algorithms to previously developed and used algorithms and models. This way an AI algorithm can be seen as a way to enhance the already developed and tested estimators.

2. Replace the previously used algorithms with a trained AI algorithm.

In article [14] a physics-constrained neural network (PCNN) has been introduced. This approach merges the well established and developed physical models with the advantages of a neural network (NN). Here the created network is capable of learning the coefficients of a single track vehicle model that can predict the vehicle's movement. The main advantage of this application is that by using the known dynamic and physical properties of a vehicle, the coefficients can be determined faster and more precisely than if there were no established constrains.

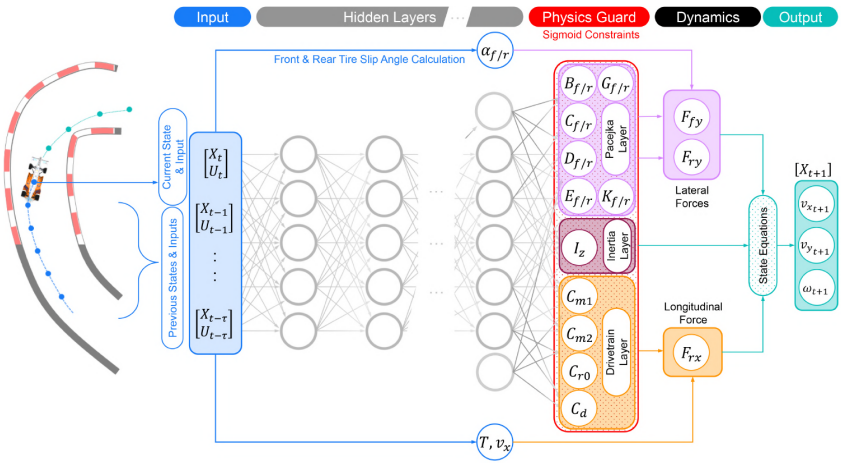


Figure 2.10: Physics constrained neural network from [14]

A similar approach was investigated in [15] where a physics embedded neural network in combination of a vehicle model was developed where the previously discussed advantages were exploited. The outcome of this investigation was the performance shift from the baseline neural network model showing faster learning speed, higher accuracy and better generalization.

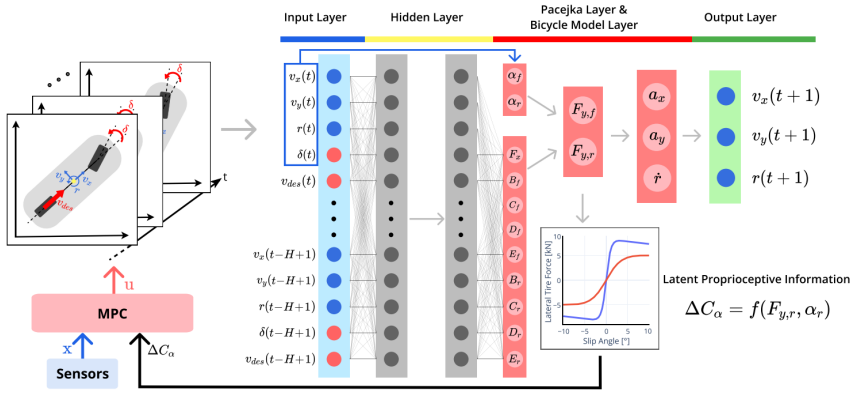


Figure 2.11: Physics embedded neural network for state estimation from [15]

An other approach is to combine AI algorithms with Kalman filtering to enhance state estimation [16], where a method for combining Long-Short Term Memory network (LSTM), Transformers and Expectation-Maximization - Kalman Filter (EM-KF) is presented. This combined approach of TL-KF (Transformers, LSTM - Kalman filter) is capable of capturing long term dependencies and model time-series data. The structure of the developed model can be seen on figure 2.12

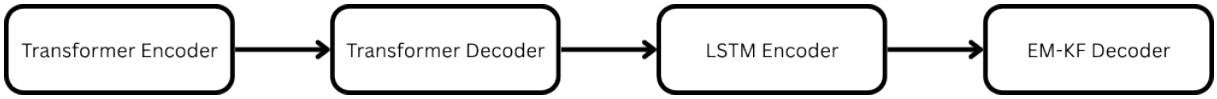


Figure 2.12: TS-KF model from [16]

2.4 Conclusions

During the literature review a comprehensive overview was presented about the used and developed vehicle speed estimators. It is visible that the well established estimators, like different model based estimators with Kalman filters have been there for longer time. In the meantime, the newly emerging field of AI has a promising future in state estimation aiding the already well used estimator algortihms.

Chapter 3

Development environment

In this chapter the environment of the development will be described, where the structure of the vehicle and its sensors, the available measurement data, the possible AI based approaches for vehicle speed estimation and the used software and hardware tools will be discussed.

3.1 Available sensor data

A modern vehicle is equipped with multiple ECU-s, numerous sensors and actuators that support and enhance driving quality. When considering vehicle motion control applications the available sensors are the following:

Sensor Type	**Variables**	**Unit of Measurement**
Wheel speed sensor	$\omega_{FL}, \omega_{FR}, \omega_{RL}, \omega_{RR}$	[°/s]
Drive torque sensor	$M_{FL}, M_{FR}, M_{RL}, M_{RR}$	[Nm]
Acceleration from IMU	a_x, a_y, a_z	[m/s^2]
Angular rates from IMU	$\dot{\varphi}$, $\dot{\theta}$, $\dot{\psi}$	[°/s]
Brake pressure sensor	$P_{FR}, P_{FL}, P_{RR}, P_{RL}$	[bar]
Road wheel angles	$\delta_{FR}, \delta_{FL}, \delta_{RR}, \delta_{RL}$	[°]

Table 3.1: Overview of sensor types, variables, and units

It is important to note here that the it is assumed that the road wheels on one axle will be assigned same angle. Therefore the a road wheel angles ($\delta_{FR}, \delta_{FL}, \delta_{RR}, \delta_{RL}$) will be simplified to a two distinct values, where the front wheels will have the same angle ($\delta_{FR} = \delta_{FM}$ and $\delta_{FL} = \delta_{FM}$) and the rear wheels will have the same angle ($\delta_{RR} = \delta_{RM}$ and $\delta_{RL} = \delta_{RM}$). From this point the front and rear wheel angles will be represented as rwa_{FM} and rwa_{RM} respectively.

GNSS receivers can be also considered as precise sensors that are available during the development phase but not on the vehicles from the production line in order to keep the wider availability. Therefore the calculated longitudinal and lateral speeds from the GNSS

measurements will only serve for reference, validation, testing and training purposes. The mentioned sensor units can be seen on figure 3.1.

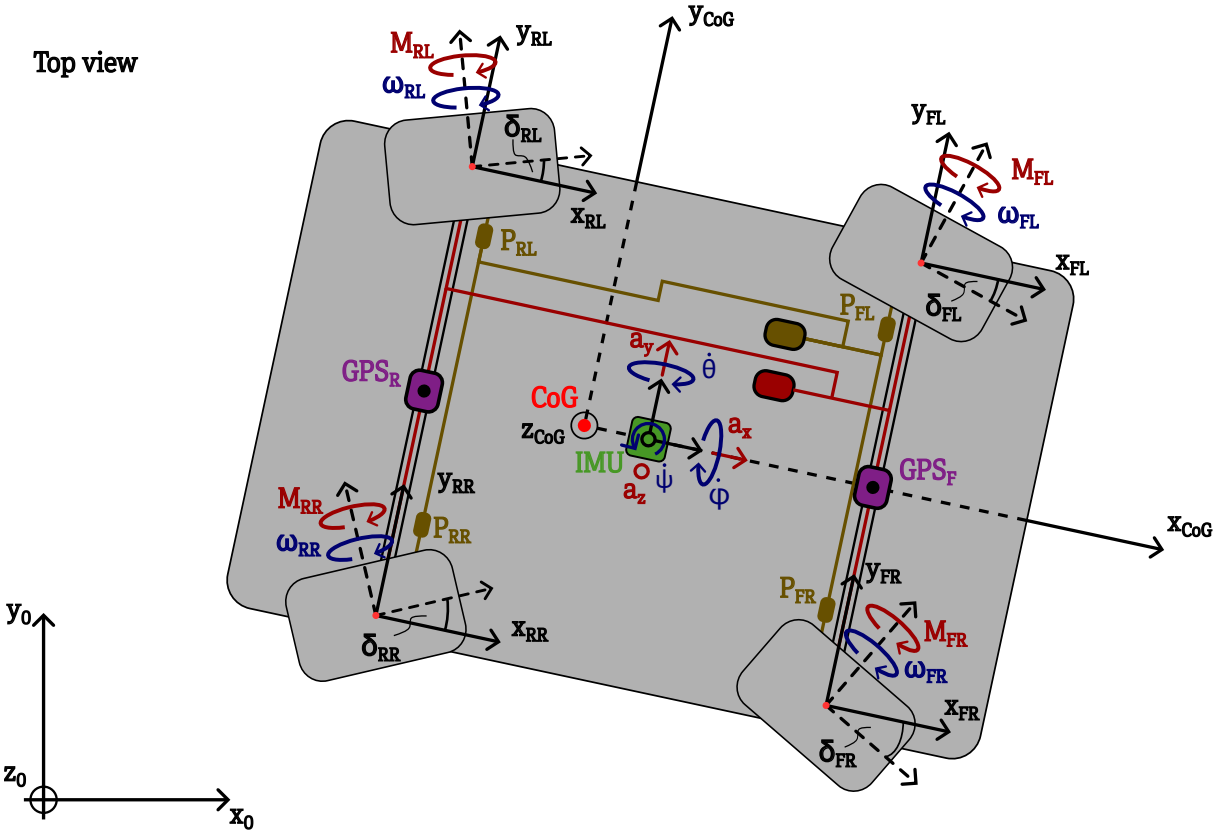


Figure 3.1: Available sensor data for speed estimation

The structure of the available sensor on the CAN bus data that was used in the further development is the following. Sampling time is 10 $[ms]$ for all the sensors except the speed values provided by the GNSS receivers, that is 100 $[ms]$. Above this, the sensor data is collected in a large enough buffer to provide all the values at the same time. This way at every 10 $[ms]$ all the inputs will arrive at the same time to the estimator. The speeds from the GNSS receivers will be handled separately from base structure. The structure can be seen in table 3.2:

time	a_x	ω_{FR}	ω_{FL}	ω_{RR}	ω_{RL}	M_{FR}	M_{FL}	M_{RR}	M_{RL}	a_y	a_z	$\dot{\varphi}$	$\dot{\theta}$	$\dot{\psi}$	...
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	...
6.220	0.2819	31.3643	31.3791	31.5755	31.5894	0	0	19.087	19.087	0	0	0	0	0.2329	...
6.222	0.2819	31.3643	31.3791	31.5755	31.5894	0	0	19.087	19.087	0	0	0	0	0.2349	...
6.224	0.2819	31.3643	31.3791	31.5755	31.5894	0	0	19.087	19.087	0	0	0	0	0.2369	...
6.226	0.2819	31.3643	31.3791	31.5755	31.5894	0	0	19.087	19.087	0	0	0	0	0.2389	...
6.228	0.2819	31.3643	31.3791	31.5755	31.5894	0	0	19.087	19.087	0	0	0	0	0.2409	...
6.230	0.2893	31.511	31.5304	31.7222	31.7406	0	0	19.087	19.087	0	0	0	0	0.2429	...
6.232	0.2893	31.511	31.5304	31.7222	31.7406	0	0	19.087	19.087	0	0	0	0	0.2449	...
6.234	0.2893	31.511	31.5304	31.7222	31.7406	0	0	19.087	19.087	0	0	0	0	0.2469	...
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	...

Table 3.2: Example of available data structure with 2$[ms]$ sampling time

3.2 Acquiring measurement data for development and testing

It is important to note that all the measurement data that was used during the development and testing phase of the estimators were recorded from a real vehicle. The vehicle used for the measurements was a luxury sedan. Apart from using the vehicle's built in sensors, two high precision GNNS receivers were installed on the roof of the vehicle to provide a reference speed, that can be used for validation and training purposes.

The measurement data was recorded in multiple test cases, where different driving scenarios were covered. The scenarios included basic maneuvers that were the following: accelerating until a target longitudinal acceleration is reached, decelerating until a target longitudinal acceleration is reached, steering until a target lateral acceleration is reached and a sine wave-like steering where the steering wheel is turned left and right continuously. These maneuvers were tested at different speeds to cover a wide range of operation. In addition the first two maneuvers can be split into two subgroups based on a change in a parameter that influenced the vehicle dynamics. With this in mind the test cases can be groupped into 6 different categories.

All together 43 measurements were recorded, where each measurement file contains a specified maneuver with a duration between 15 and 25 $[s]$. The sampling time of the recorded data is 2 $[ms]$ but was downsampled to 10 $[ms]$ during the development phase as the GNSS receiver only provided speed values with this sampling time.

In order to be able to train, validate and test the developed estimators, the recorded measurement data was split into three different sets. The training set contains 31 measeurement files, the validation set and the test set contain 6-6 measurement files. The splitting was done in a way that all the different maneuvers are represented in each set.

Besides the sensor data from the vehicle and the high precision GNSS receivers, the speed estimates from the vehicle's built in ECU were also recorded. These speed values will also serve as a reference for the performance of the developed estimators.

Furthermore the recorded measurement data was normalized before using it for training and testing. The normalization was done with the following formula:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}, \tag{3.1}$$

where x is the original variable, x_{norm} is the normalized variable, x_{min} is the minimum value of the variable in the dataset and x_{max} is the maximum value of the variable in the dataset. This was done to ensure that all the input variables are in the range of [0...1], which helps the training process of the AI models in way that no input variable dominates the learning process due to its larger scale. The normalization was done using min-max scaling, where the minimum and maximum values for each variable were determined from

the dataset. The minimal and maximal values used for normalization can be seen in table 3.3 for all the variables. This is important due to the limitations of the models as it will only be able to provide accurate estimates within the range of the training data.

Variable	Min.	Max.
a_x	0 [m/s^2]	10.6703 [m/s^2]
a_y	0 [m/s^2]	11.5285 [m/s^2]
a_z	0 [m/s^2]	0.2543 [m/s^2]
ω_{FL}	0 [°/s]	87.4844 [°/s]
ω_{FR}	0 [°/s]	87.4063 [°/s]
ω_{RL}	0 [°/s]	87.5459 [°/s]
ω_{RR}	0 [°/s]	87.6406 [°/s]
M_{FL}	0 [Nm]	184.1236 [Nm]
M_{FR}	0 [Nm]	184.1236 [Nm]
M_{RL}	0 [Nm]	324.8717 [Nm]

Variable	Min.	Max.
M_{RR}	0 [Nm]	324.8717 [Nm]
$\dot{\varphi}$	0 [°/s]	0.1504 [°/s]
$\dot{\theta}$	0 [°/s]	0.3755 [°/s]
$\dot{\psi}$	0 [°/s]	1.2396 [°/s]
rwa$_{FM}$	-0.5865 [°]	0.5979 [°]
rwa$_{RM}$	-0.0545 [rad]	0.0615 [°]
P_{FL}	0 [bar]	220.3667 [bar]
P_{FR}	0 [bar]	217.8333 [bar]
P_{RL}	0 [bar]	184.6842 [bar]
P_{RR}	0 [bar]	180.2632 [bar]

Table 3.3: Min and max values used for normalization of the sensore data

The validation measurements for estimating vehicle speed can be seen in figure 3.2 where different driving scenarios are represented. The numbers correspond to the different driving maneuvers that were described earlier in this section. It needs to be mentioned that the term "Snow mode" refers to a driving mode where the rear wheel steering is deactivated.

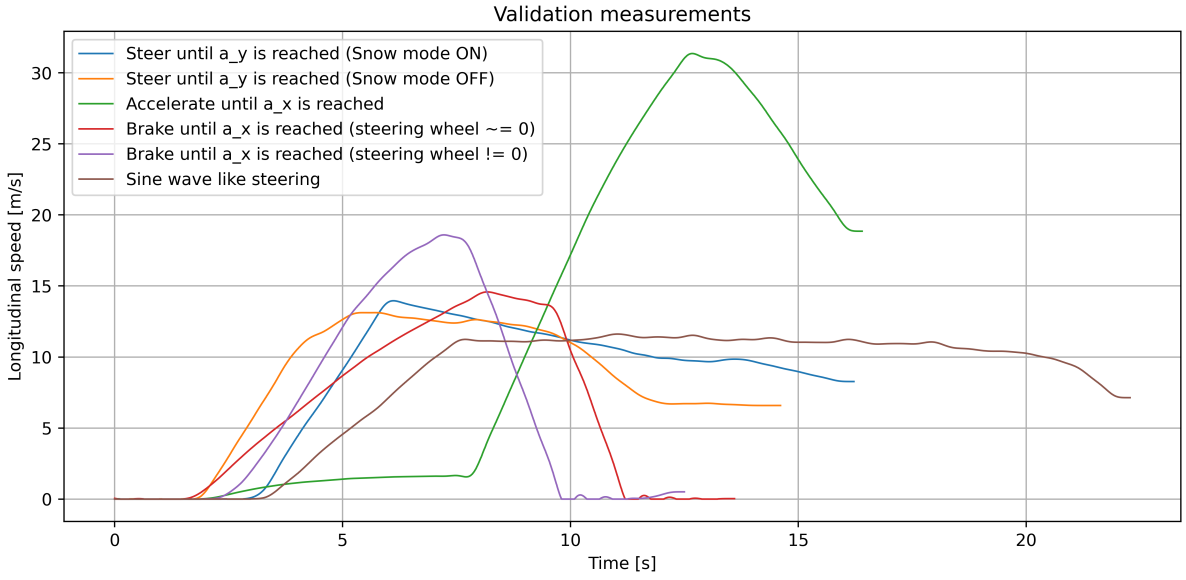


Figure 3.2: Recorded validation measurements for vehicle speed estimation

3.3 Ways to use AI to estimate vehicle speed

Before advancing to the possible algorithms for vehicle speed estimation using AI, some high level requirements shall be set to be able to validate the results. The requirements are the following:

1. The developed estimator shall provide more accurate vehicle speed values than a model based vehicle speed estimator.

2. The developed estimator shall be tested and validated with data that was not used for training purposes.

3. The developed estimator shall be trained with data that covers the range of use of a vehicle.

4. The developed estimator shall be applicable in real time usage in a test vehicle.

With these requirements in mind the possible solutions can be discussed. The algorithms can be categorized into two groups. Solutions where the vehicle speeds are directly estimated by the AI algorithm from the sensor input data (1) and where the AI algorithm is combined with an already existing estimator (2). Both of these solutions offer great potential in estimating vehicle speed.

The general structure of a vehicle speed estimator in a vehicle's control system can be seen in figure 3.3. The estimator receives the sensor data from the CAN bus and provides the estimated vehicle speed to the control algorithm that uses it for further calculations.

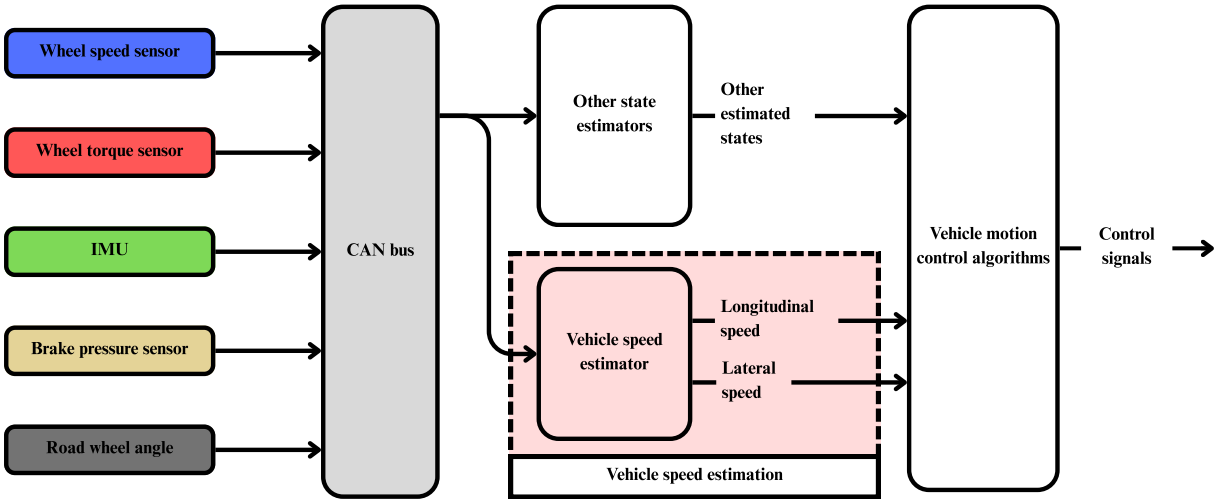


Figure 3.3: Vehicle speed estimation in a control algorithm

3.3.1 Direct estimation

When directly estimating any state of the vehicle, the structure is the following. The trained AI model receives the sensor data as inputs and outputs the state for which it was trained. This thesis the focus is on estimating the longitudinal speed of the vehicle.

The question now arises what models are suitable for this application? The possible models to use in this scenario are models that can handle time series data well and can estimate an output based on the sequence it was provided with. The investigated models that will be described in section 4 are the following:

- Basic/simple Recurrent Neural Network (RNN)
- Long-short Term Memory (LSTM)
- Gated Recurrent Unit (GRU)
- Transformer for time series
- Temporal Convolutinal Network (TCN)

The teaching phase of these models will be relatively similar. All models will be taught on previously recorded sensor data coming from vehicle simulations and real-life vehicle tests. These recorder datasets will be extended with a longitudinal and lateral speed values calculated from the GNSS measurements. This way every input will be labeled with the desired output.

3.3.2 Combination of existing estimators with AI model

As it can be seen in the literature review that there are already existing applications of combining an AI model with an already established state estimator. The key advantages of this approach could be improved robustness, adaptive parameter tuning of the applied filter with implementing AI and non-linear error compensation.

Some possible implementations can be the following:

- Residual learning:
  - A neural network predicts the residual error from a standard Extended Kalman Filter (EKF) applied for vehicle speed estimation. The final speed will result from the combination of the estimated and the neural network's value for the residual.
- Neural network aided Kalman filter:
  - The parts of the Kalman filter are replaced with trained modules

  - E.g when the predict module is replaced by a neural network that determines the next state.

- Adaptive gain scheduler:

  - The AI module dynamically tunes the noise covariance of the implemented Kalman filter based on the CAN signals.

3.3.3 Advantages and disadvantages of AI application

Prior to moving on to describing the developed models and estimators it is important to state the advantages and disadvantages when applying an AI model to a vehicle's control system. In case of a vehicle that is designed to be in contact (to transport or be driven by) with humans, the developed control system has to comply with specific international standards (e.g ISO 26262) that assures the safety of the system.

The main advantage when implementing AI to a vehicle control system is the possibility of capturing non-linear relationships between the CAN signals to estimate an arbitrary state. This property makes it possible to establish complex relationships between state variables without the knowledge of more complicated physical formulas.

On the other hand, there are some disadvantages that need to be considered. An implemented AI can be considered as a black box where the inner operation makes it difficult to validate from safety relevant perspective. This also creates a part of the system that lacks transparency and explainability. An other disadvantage may arise from missing out on data quality and availability. Resource constraints in embedded systems may also block usability if the created model uses more computational power or memory than the available amount.

Chapter 4

Direct estimation with AI algorithms

In this chapter the chosen AI models that are suitable for vehicle speed estimation will be discussed. First, three different versions of the recurrent neural network (RNN) will be presented including the vanilla recurrent neural network (RNN), the long short-term memory network (LSTM) and the gated recurrent unit network (GRU). Then the temporal neural network (TCN) will be introduced as a convolutional alternative to the recurrent networks. Finally, transformer based models for time series will be discussed as another possible solution for vehicle speed estimation.

4.1 General development

Throughout this chapter different AI models will be presented for a sequence modeling task, that is estimating the vehicle speed from a time series of sensor measurements. The main goal is to find a a function that maps the input sensor data sequence to the desired vehicle speed value as accurately as possible.

$$\hat{y}_t = f(x_{t-n}, x_{t-n+1}, ..., x_t), \tag{4.1}$$

where $\hat{y}_t$ is the estimated vehicle speed at time step t, $x_{t-n}, x_{t-n+1}, ..., x_t$ are the input sensor measurements from time step $t-n$ to t, and f is the function approximated by the AI model.

The development of the different AI models for direct vehicle speed estimation will follow a similar structure in order to be able to compare the results and performance of the different architectures.

Initially, the train, validation and test datasets are the identical for all training sequences to ensure the comparibility of the architectures.

Next, the parameters of the architectural structure will be defined. The permutation stemming from all the possible parameter values cannot be tested due to the high computational and time cost. Therefore a grid like search will be performed where the most

influential parameters will be varied and the best performing combination will be chosen for further testing.

Finally a testing procedure will be carried on all the models using the previously unseen test dataset. The estimated vehicle speed values will be compared to the reference values where a set of metrics will be calculated to determine the performance of the different architectures.

The steps of the development procedure will be discussed in more detail in the section 4.2.1 containing the vanilla RNN model.

4.2 Recurrent Neural Network - RNN

4.2.1 Vanilla RNN

Theoretical background

Contrary with regular the feedforward neural networks where the output is determined only by the current values of the inputs, the Recurrent Neural Network (RNN) is designed in a way that after each step the information is fed back to the network. This can be seen in figure 4.1. This way the output will be dependent on the previous inputs as well and the model will have a memory like property giving the possibility of handling temporal data.

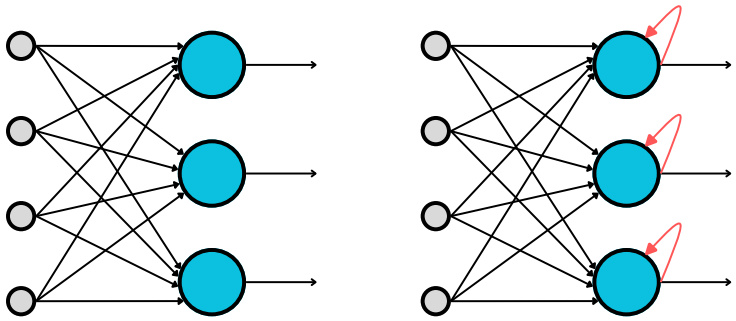


Figure 4.1: Feedforward nn (left) compared to recurrent nn (right)

The fundamental building blocks of a RNN are the recurrent units (RU) or RNN cells. These units hold an inner state that connects the in- and outputs. The inner states maintain information about previous inputs in a sequence. To understand how does the network's architecture function, let it be unfolded in time. This can be seen on figure 4.2.

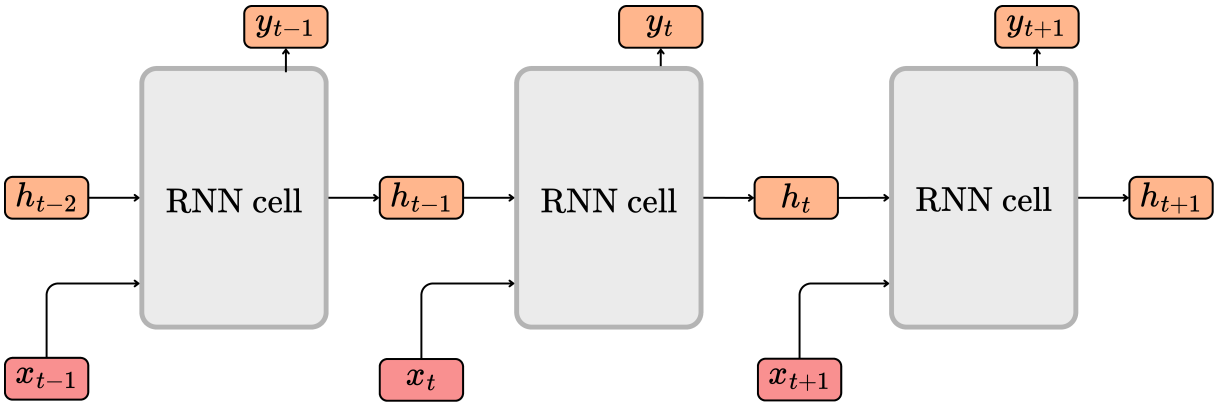


Figure 4.2: Unfolding the architecture of a RNN in time

The dimensions of the vectors in the RNN are the following:

- Input vector: $\mathbf{x}_t \in \mathbb{R}^{d_x}$, where d_x is the size of the input vector.
- Hidden state: $\mathbf{h}_t \in \mathbb{R}^{d_h}$, where d_h is the number of hidden units in the RNN layer.
- Output vector: $\mathbf{y}_t \in \mathbb{R}^{d_y}$, where d_y is the size of the output vector.

It can be seen that at each time step the output vector ($\mathbf{y}_t$) is determined by not only the input vector ($\mathbf{x}_t$), but the state of the hidden layer in the previous time step ($\mathbf{h}_{t-1}$) as well. The inner structure of the RNN cell can be seen on figure 4.3.

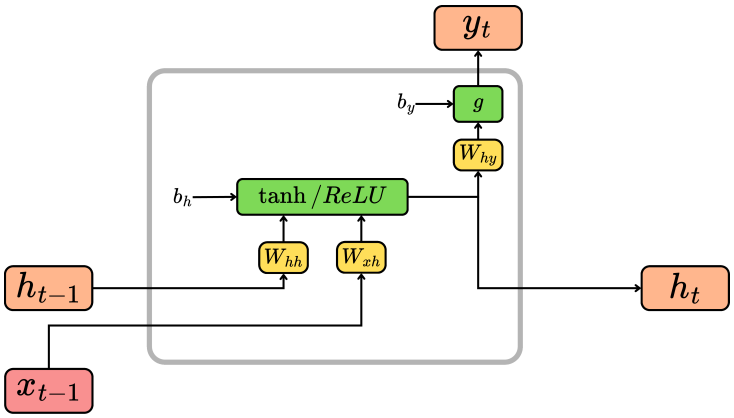


Figure 4.3: Recurrent Unit in the RNN

The dimensions of the weight matrices and bias vectors are the following:

- Weight matrix for the input vector: $\mathbf{W}_{xh} \in \mathbb{R}^{d_x \times d_h}$
- Weight matrix for the state vector: $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$
- Weight matrix for the output vector: $\mathbf{W}_{hy} \in \mathbb{R}^{d_h \times d_y}$
- Bias vector for the hidden layer: $\mathbf{b}_h \in \mathbb{R}^{d_h}$

- Bias vector for the output layer: $\mathbf{b}_y \in \mathbb{R}^{d_y}$

The governing equations for the RNN cell are the following:

1. Hidden state update 4.2: The hidden state at time step t is computed using the current input vector ($\mathbf{x}_t$) and the previous hidden state ($\mathbf{h}_{t-1}$) through a weighted sum followed by an activation function (e.g., tanh or ReLU).

$$\mathbf{h}_t = f(\mathbf{W}_{xh} \cdot \mathbf{x}_t + \mathbf{W}_{hh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_h) \tag{4.2}$$

2. Output computation 4.3: The output vector at time step t is determined from the current hidden state ($\mathbf{h}_t$) using a linear transformation followed by an activation function (depending on the task, e.g., softmax for classification or identity for regression).

$$\mathbf{y}_t = g(\mathbf{W}_{hy} \cdot \mathbf{h}_t + \mathbf{b}_y) \tag{4.3}$$

To determine the weights and biases of the network, backpropagation through time (BPTT) is used, that is an extended version of the backpropagation used in feedforward neural networks (FNNs). This method is based on the error between the determined output of the network and the desired/labeled output at time step t. BPTT involves two major steps: forward pass and backward pass. During the forward pass the inputs are passed through the network to compute the outputs and the loss, while during the backward pass the gradients of the loss with respect to the weights are calculated to update them accordingly. The steps of BPTT are the following:

1. Forward pass

   - During the forward pass the sequence of inputs form $t = 1$ to $t = n$ passes through the network, where n is the length of the sequence.
   - The output vector $\mathbf{y}_t$ is determined at every timestep with the use of the previously described formulas.
   - The loss is determined at every timestep between the desired ($\mathbf{y}_t^d$) and the estimated ($\mathbf{y}_t$) output.
   - The loss function is given by the mean squared error (e.g. MSELoss):

$$L(\mathbf{y}, \mathbf{y}^d) = \frac{1}{t} \sum_{t=1}^{n} (\mathbf{y}_t - \mathbf{y}_t^d)^2 \tag{4.4}$$

2. Backward pass

   - During the backward pass the gradients of the loss function $L(\mathbf{y}, \mathbf{y}^d)$ are calculated with respect of the weights of the network ($\mathbf{w}_{xh}, \mathbf{W}_{hh}, \mathbf{W}_{hy}$).

- W.r.t the weight matrices the derivatives of the loss functions can be determined as:

$$\frac{\partial \mathbf{L}}{\partial \mathbf{W}_{xh}} = \sum_{t=1}^{n} \frac{\partial \mathbf{L}}{\partial \mathbf{h}_t} \cdot \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{xh}} \tag{4.5}$$

$$\frac{\partial \mathbf{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{n} \frac{\partial \mathbf{L}}{\partial \mathbf{h}_t} \cdot \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} \tag{4.6}$$

$$\frac{\partial \mathbf{L}}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^{n} \frac{\partial \mathbf{L}}{\partial \mathbf{y}_t^d} \cdot \frac{\partial \mathbf{y}_t^d}{\partial \mathbf{W}_{hy}} \tag{4.7}$$

- The gradient values inform about the direction (increase or decrease) and the magnitude of the change in the weight matrices.
- These gradients can then be used for updating the values of each weight matrix with the help of the chosen optimizer (e.g. SGD, Adam).

Common issues when applying RNNs are exploding and vanishing gradients during the backward pass. Both of these refer to the magnitude of the calculated gradients: $\left\| \frac{\partial \mathbf{L}}{\partial \mathbf{W}} \right\|$. Exploding gradient problem stems from the gradient being too high which is a consequence of the learning rate being to big. On the other hand vanishing gradient refers to a gradient being near zero, which leads to slow learning.

Implementation and results

The objective is to find the best performing RNN architecture for estimating the vehicle speed from the given input data. For this the following paramaters were taken into account: number of inputs, hidden size and number of layers. The number of inputs refers to the dimension of the input vector (d_x) where every element of the vector corresponds to a different sensor measurement. The hidden size refers to the dimension o the hidden state vector (d_h) and it determines how much information the hidden state can store. Finally the number of layers refers to how many RNN layers are stacked on top of each other. An example for a 2-layer RNN can be seen on figure 4.4. Another parameter that was considered, but is not part of the architecture was the sequence length. This parameter defines how many time steps the network will look back when estimating the current vehicle speed.

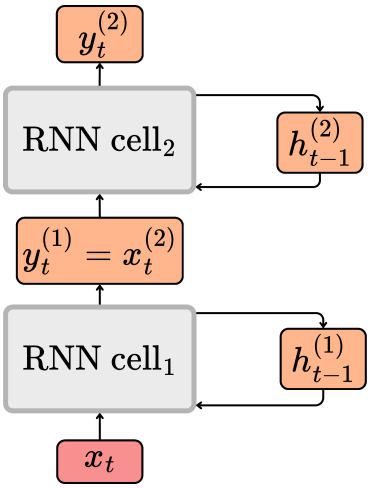


Figure 4.4: 2-layer RNN architecture

The initial values for the mentioned hyperparameters can be seen in table in 4.1. The overall number of trained models in the first batch was $32 \cdot 2 = 64$. The values were chosen based on common practices in literature and previous experience with RNN-s [17]. A grid search was performed over the hyperparameter space to find the best combination of values that yield the lowest validation loss. The training was done using the Adam optimizer with a learning rate of 0.001 and a batch size of 128. The models were trained for 100 epochs, and early stopping was implemented to prevent overfitting.

Hyperparameters	**Possible values**
Number of inputs (d_x)	23; 12
Hidden size (d_h)	64; 96; 128; 192
Number of layers	1; 2
Sequence length	50; 100; 150; 200

Table 4.1: Hyperparameter ranges for RNN architecture search

First, the effect of the number of inputs was examined with the intent of reducing the complexity of the model. The available sensor measurements at each time step can be found in table 3.1. When considering all the available sensor measurements the input vector has a dimension of 23. However, the number of inputs can be reduced to a subset of all the available values when considering only the most relevant ones. Table 4.2 contains the subset of the inputs that was used. Within the table x marks the inputs that were used and - marks the ones that were omitted in the subset.

Sensor measurement	**ID 1**	**ID 2**	**Reason of omiting**
Wheel speed FL - ω_{FL}	x	x	-
Wheel speed FR - ω_{FR}	x	x	-
Wheel speed RL - ω_{RL}	x	x	-
Wheel speed RR - ω_{RR}	x	x	-
Drive torque FL - M_{FL}	x	x	-
Drive torque FR - M_{FR}	x	-	The drive torques for the wheels on one axle is equivalent due to open differential $\rightarrow$ no additional info compared to M_{FL}
Drive torque RL - M_{RL}	x	x	-
Drive torque FL - M_{RR}	x	x	The drive torques for the wheels on one axle is equivalent $\rightarrow$ no additional info compared to M_{RL}
Long. acceleration - a_x	x	x	-
Lat. acceleration - a_y	x	x	-
Vert. acceleration - a_z	x	-	Vertical acceleration is not significant as the vehicle measurements were made on a plane
Roll rate - $\dot{\varphi}$	x	x	Roll rate is significant in a planar measurement but was omitted to imitate a 2D vehicle
Pitch rate - $\dot{\theta}$	x	-	Pitch rate is not significant as the vehicle measurements were made on a plane surface
Yaw rate - $\dot{\psi}$	x	x	-
Brake pressure FR - P_{FR}	x	x	-
Brake pressure FL - P_{FL}	x	-	Brake pressures for the wheels on one axle is equivalent throughout the measurements $\rightarrow$ no additional info compared to P_{FR}
Brake pressure RR - P_{RR}	x	x	-
Brake pressure RL - P_{RL}	x	-	Brake pressures for the wheels on one axle is equivalent throughout the measurements $\rightarrow$ no additional info compared to P_{RR}
Road wheel angle - rwa_{FM}	x	x	-
Road wheel angle - rwa_{RM}	x	x	-

Table 4.2: The set of sensor measurements used as inputs

The hidden sizes, the number of layers and the sequence lengths were varied and a permutation of the possible values was tested. Two different input sets were used: one with all the available sensor measurements (23 inputs) and one with the reduced set of sensor measurements (12 inputs). This way the effect of the number of inputs on the performance of the model could be examined.

The chosen performance metrics for evaluating the models and the estimators were the root mean squared error (RMSE) and the mean absolute error (MAE). Apart from that, and absolute maximum error (AMaxE) metric was also calculated to see the worst case performance of the models. Furthermore a percentage within threshold (PWT) metric was also defined to see how many of the estimations were within a certain error threshold. The PWT metric is defined as follows:

$$PWT = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(|y_i^d - y_i| < \epsilon) \times 100\%, \tag{4.8}$$

where N is the total number of estimations, y_i is the estimated value, y_i^d is the desired value, ϵ is the error threshold (e.g., 1 km/h), and $\mathbb{I}$ is the indicator function that returns 1 if the condition is true and 0 otherwise. Throughout the development two different thresholds were considered: 0.3 km/h and 0.4 km/h. These will be denoted as PWT 0.3 and PWT 0.4 in the following tables.

After the training and validation procedure all the models were tested on the unseen test dataset. The code using for training and validating can be found at `AI_training/3_code/training_RNN.py`. The 3-3 best performing architectures were chosen based on the root mean squared error (RMSE) metric. The 3 best performing architectures with the two input sets can be seen in table 4.3. The results that can be found in table 4.4 show that the model with 23 inputs performs slightly better than the one with 12 inputs, but the difference is not significant, therefore the reduced input set can be used to lower the complexity of the model without sacrificing performance.

	Input ID	**Sequence Length**	**Hidden size**	**Number of layers**
1	1	100	96	1
2	1	50	128	1
3	1	100	192	2
1	2	150	192	1
2	2	50	96	1
3	2	150	64	1

Table 4.3: Results of the best performing RNN architecture on the test dataset

	RMSE [$\frac{m}{s}$]	MAE [$\frac{m}{s}$]	AMaxE [$\frac{m}{s}$]	PWT 0.3 [%]	PWT 0.4 [%]
1	0.14356	0.08236	1.42788	96.13440	97.35484
2	0.14701	0.08151	1.53784	95.48518	96.87399
3	0.15065	0.08569	1.57739	95.16015	96.56048
1	0.15388	0.08794	2.18230	95.06897	96.59202
2	0.15650	0.08913	1.63804	95.58468	96.92083
3	0.16051	0.08786	1.74411	95.42522	96.88414

Table 4.4: Performance metrics of the best performing RNN architectures on the test dataset

It can be seen that the number of inputs has a little effect on the performance of the models, but not significant enough to justify the increased complexity of the model. A design decision from this point on was to continue the development with the reduced input set (12 inputs) as it yielded similar results while having a lower complexity.

After obtaining these results another grid search was performed around the 3 best performing architectures to see if further improvements could be achieved by finetuning the hyperparameters. The hyperparameter ranges for the finetuning can be seen in table 4.5.

Around 1st best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	48, 64, 80
Number of layers	1, 2
Sequence length	90, 100, 110
Around 2nd best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	80, 96, 112
Number of layers	1, 2
Sequence length	40, 50, 60
Around 3d best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	48, 64, 80
Number of layers	1, 2
Sequence length	140, 150, 160

Table 4.5: Batch 2: Hyperparameter ranges for RNN architecture search

Following the new training and validation procedure, that conducted 54 new models, the best performing architectures were again tested on the unseen test dataset. The results of the best performing architecture can be seen in table 4.6 and 4.7. It can be seen that the finetuning of the hyperparameters led to a slight improvement in the performance of

the models. The estimates of the best performing RNN architecture for the validation dataset can be seen on figure 4.5. On the figure on the left indicates the valiadation where the RNN performed the worst while the figure on the right indicates the validation where the RNN performed the best.

	Input ID	**Sequence Length**	**Hidden size**	**Number of layers**
1	2	40	96	1

Table 4.6: Results of the best performing RNN architecture on the test dataset after finetuning

	RMSE $[\frac{m}{s}]$	**MAE** $[\frac{m}{s}]$	**AMaxE** $[\frac{m}{s}]$	**PWT 0.3 [%]**	**PWT 0.4 [%]**
1	0.12640	0.06590	1.49319	96.91017	97.52393

Table 4.7: Performance metrics of the best performing RNN architectures on the test dataset after finetuning

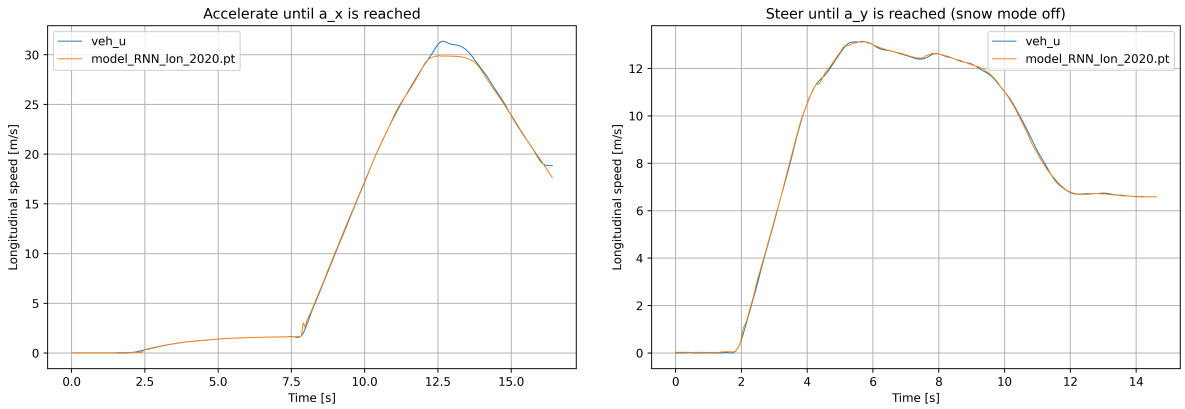


Figure 4.5: Predictions of the best performing RNN architecture on the validation dataset after finetuning (left: worst case, right: best case)

It can be seen that the after the finetuning the performance metrics have improved. This indicates that the finetuning of the hyperparameters was successful and led to an improvement in the performance of the model.

4.2.2 Long Short-Term Memory - LSTM

Theoretical background

The Long Short-Term Memory (LSTM) networks build on the structure of the RNN-s but introduce additional modifications to improve the performance in long term dependencies when handling sequential data. The main difference between the two structures are the introduction of memory cells and a series of so called gating mehanisms. The output of the LSTM unit is determined not only by the current input and the previous hidden state,

as it was with the conventional RNN-s, but also by the newly introduced cell state that carries information across the time steps. The structure of an LSTM network unfolded in time can be seen on figure 4.6, where x_t is the input vector, h_t is the hidden state vector, C_t is the cell state vector and y_t is the output vector at time step t.

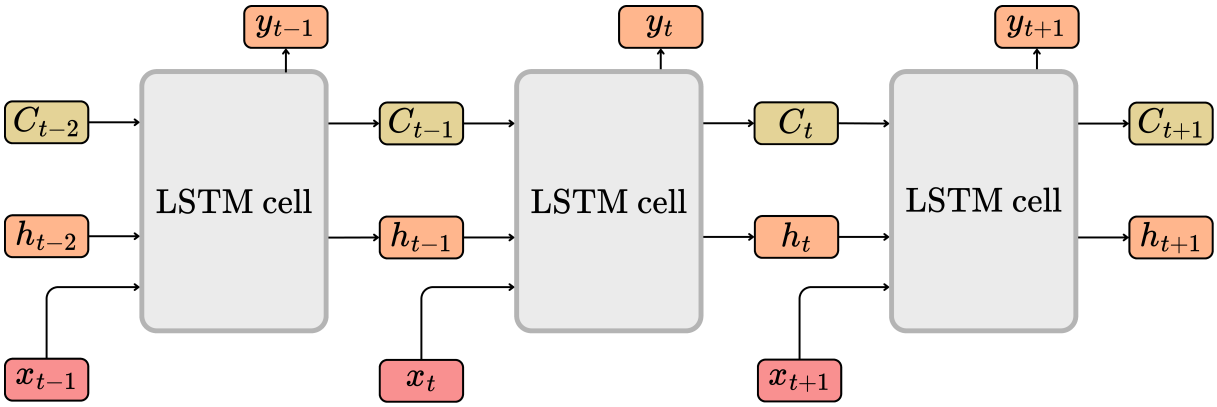


Figure 4.6: LSTM sequence unfolded in time

The dimensions of the previously mentioned vectors are the following:

- Input vector: $\mathbf{x}_t \in \mathbb{R}^{d_x}$, where d_x is the size of the input vector.
- Hidden state: $\mathbf{h}_t \in \mathbb{R}^{d_h}$, where d_h is the number of hidden units in the LSTM layer.
- Cell state: $\mathbf{C}_t \in \mathbb{R}^{d_h}$, that has the same dimension as the hidden state vector.
- Output vector: $\mathbf{y}_t \in \mathbb{R}^{d_y}$, where d_y is the size of the output vector.

I want to note here that I have not included the dimension of the batch size B for simplicity. Here I only consider the inputs to be a single sequence, as it will be when using the netowork for estimating the vehicle speed. When considering trainging with batches of sequences, the dimensions would be: $\mathbf{x}_t \in \mathbb{R}^{B \times d_x}$, $\mathbf{h}_t \in \mathbb{R}^{B \times d_h}$, $\mathbf{C}_t \in \mathbb{R}^{B \times d_h}$, $\mathbf{y}_t \in \mathbb{R}^{B \times d_y}$.

The white-box represetation of the LSTM cell can be seen on figure 4.7, where the weight matrices and the bias vectors are also presented. The dimensions of these vectors are the following:

- Weight matrices for the input vector: $\mathbf{W}_{xf}, \mathbf{W}_{xi}, \mathbf{W}_{xc}, \mathbf{W}_{xo} \in \mathbb{R}^{d_x \times d_h}$
- Weight matrices for the state vectors: $\mathbf{W}_{hf}, \mathbf{W}_{hi}, \mathbf{W}_{hc}, \mathbf{W}_{ho} \in \mathbb{R}^{d_h \times d_h}$
- Weight matrix for the output vector: $\mathbf{W}_{hy} \in \mathbb{R}^{d_h \times d_y}$
- Bias vectors for the operation: $\mathbf{b}_f, \mathbf{b}_i, \mathbf{b}_c, \mathbf{b}_o \in \mathbb{R}^{d_h}, \mathbf{b}_y \in \mathbb{R}^{d_y}$

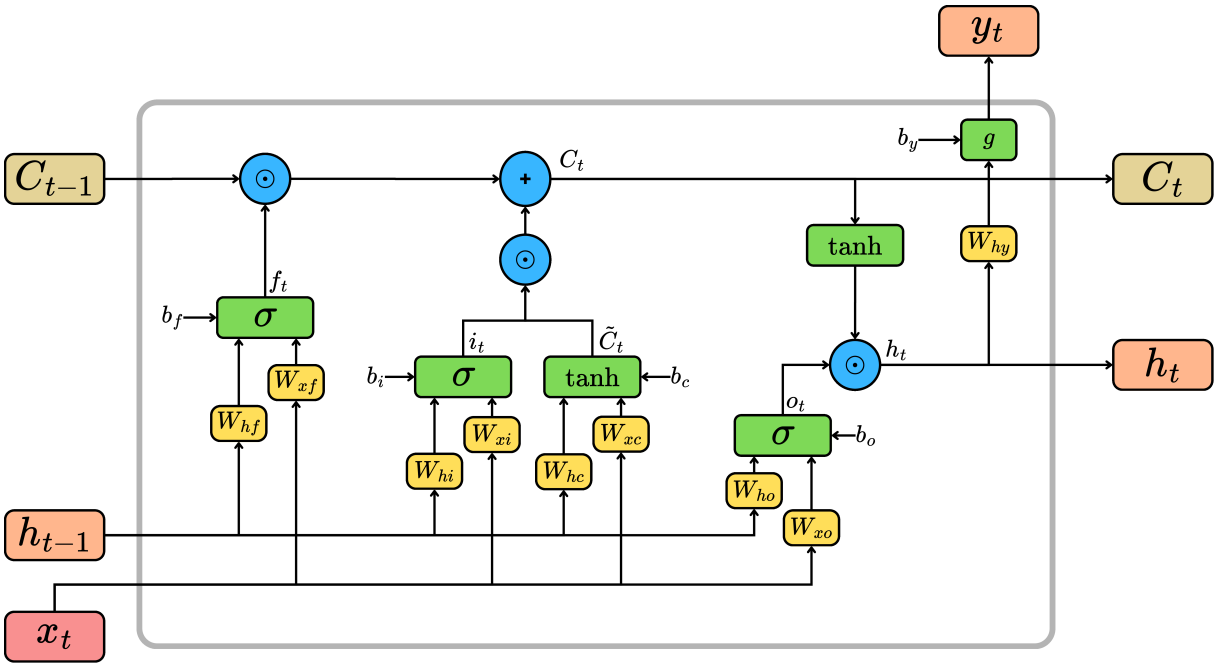


Figure 4.7: LSTM unit with the gating mechanisms

Throughout the LSTM cell the so called gate vectors control the outputs, that are the forget gate ($\mathbf{f}_t$), input gate ($\mathbf{i}_t$) and the output gate ($\mathbf{o}_t$). These are mainly functions that use either sigmoid or tanh activation functions. Furthermore the cell contains operations to determine the candidate cell state ($\tilde{\mathbf{C}}_t$), the new cell state ($\mathbf{C}_t$), the new hidden state ($\mathbf{h}_t$) and the output ($\mathbf{y}_t$). All the gates and operations can be described with the following equations:

1. Forget gate 4.9: The goal of the forget gate is to determine what information from the previous cell state ($\mathbf{C}_{t-1}$) shall be kept and what shall be discarded. This is done by using a sigmoid activation function that outputs values between 0 and 1, where 0 means "completely forget" and 1 means "completely keep".

$$\mathbf{f}_t = \sigma(\mathbf{W}_{xf} \cdot x\mathbf{x}_t + \mathbf{W}_{hf} \cdot \mathbf{h}_{t-1} + \mathbf{b}_f) \tag{4.9}$$

2. Input gate 4.10: The input gate determines what new information shall be added to the cell state. Similar to the forget gate, it uses a sigmoid activation function to decide which values to update.

$$\mathbf{i}_t = \sigma(\mathbf{W}_{xi} \cdot \mathbf{x}_t + \mathbf{W}_{hi} \cdot \mathbf{h}_{t-1} + \mathbf{b}_i) \tag{4.10}$$

3. Candidate cell state 4.11: This operation creates a vector of new candidate values ($\tilde{\mathbf{C}}_t$) that could be added to the cell state. It uses a tanh activation function to

ensure the values are between -1 and 1.

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_{xc} \cdot \mathbf{x}_t + \mathbf{W}_{hc} \cdot \mathbf{h}_{t-1} + \mathbf{b}_c) \tag{4.11}$$

4. New cell state 4.12: The new cell state ($\mathbf{C}_t$) is computed by combining the previous cell state ($\mathbf{C}_{t-1}$) modified by the forget gate and the candidate cell state ($\tilde{\mathbf{C}}_t$) scaled by the input gate. ($\odot$ denotes element-wise multiplication).

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \tag{4.12}$$

5. Output gate 4.13: The output gate determines what part of the cell state will be output as the new hidden state. It uses a sigmoid activation function to decide which parts of the cell state to output.

$$\mathbf{o}_t = \sigma(\mathbf{W}_{xo} \cdot \mathbf{x}_t + \mathbf{W}_{ho} \cdot \mathbf{h}_{t-1} + \mathbf{b}_o) \tag{4.13}$$

6. New hidden state 4.14: The new hidden state ($\mathbf{h}_t$) is computed by applying the output gate to the tanh of the new cell state.

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \tag{4.14}$$

7. Output 4.15: The output vector ($\mathbf{y}_t$) is determined from the new hidden state using a linear transformation followed by an activation function $g()$ (depending on the task).

$$\mathbf{y}_t = g(\mathbf{W}_{hy} \cdot \mathbf{h}_t + \mathbf{b}_y) \tag{4.15}$$

After defining the computational steps withing an LSTM cell, that are also the steps of the forward pass, the training process can be discussed. The objective of the training sequence is to optimize the weight matrices and the bias vectors to minimize the defined loss (L) over a dedicated training dataset, equivalent to the RNN training process.

The question still arises why is this LSTM structure better at handling long term dependencies than the conventional RNN-s? The answer lies in the constant error carousel (CEC) property of the LSTM cell. The CEC allows the error to flow unchanged through the cell state over time, which helps to mitigate the vanishing gradient problem that is common in traditional RNNs. The gating mechanisms also allow the LSTM to selectively remember or forget information, which helps to capture long-term dependencies more effectively.

Implementation and results

The implementation and training procedure of the LSTM network is similar to the one used for the RNN-s. The same reduced input set (12 inputs) was used here as well. A grid search was performed over the hyperparameter space to find the best combination of values that yield the lowest validation loss. The training was done in a similar manner as with the RNN-s. For the first batch of grid search the hyperparameter ranges are the as in table 4.1.

The results obtained after testing the created models on the same unseen test dataset . The code using for training and validating can be found at `AI_training/3_code/training_LSTM.py`. The 3 best performing architectures can be seen in table 4.8. The results that can be found in table 4.9 show that the LSTM models outperformed the RNN models, indicating that the LSTM architecture is better suited for capturing the temporal dependencies in the vehicle speed estimation task.

	Input ID	**Sequence Length**	**Hidden size**	**Number of layers**
1	2	50	192	2
2	2	100	192	1
3	2	100	96	2

Table 4.8: Results of the best performing LSTM architecture on the test dataset

	RMSE [$\frac{m}{s}$]	**MAE [$\frac{m}{s}$]**	**AMaxE [$\frac{m}{s}$]**	**PWT 0.3 [%]**	**PWT 0.4 [%]**
1	0.09454	0.05488	1.22441	97.65842	98.51403
2	0.12917	0.08065	1.23678	96.36799	97.55895
3	0.13023	0.07710	1.24691	96.35107	97.47487

Table 4.9: Performance metrics of the best performing LSTM architectures on the test dataset

After evaluating the results from the first batch it can already be seen that the LSTM models could capture the patterns in the training sequences. Following the evaluation another grid search was performed around the 3 best performing architectures to see if further improvements could be achieved by finetuning the hyperparameters. The hyperparameter ranges for the finetuning can be seen in table 4.10.

Around 1st best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	176, 192, 208
Number of layers	1, 2, 3
Sequence length	40, 50, 60
Around 2nd best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	176, 192, 208
Number of layers	1, 2
Sequence length	90, 100, 110
Around 3d best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	80, 96, 121
Number of layers	1, 2, 3
Sequence length	90, 100, 110

Table 4.10: Batch 2: Hyperparameter ranges for LSTM architecture search

Following the training and evaluation of the models from the second batch of grid search, the best performing architecture (4.11) was selected based on the RMSE metric on the test dataset. The results of this best model can be seen in table 4.12. The best model's estimates for the validation dataset can be seen on figure 4.8.

	Input ID	**Sequence Length**	**Hidden size**	**Number of layers**
1	2	50	192	2

Table 4.11: Results of the best performing LSTM architecture on the test dataset after finetuning

RMSE [$\frac{m}{s}$]	**MAE [$\frac{m}{s}$]**	**AMaxE [$\frac{m}{s}$]**	**PWT 0.3 [%]**	**PWT 0.4 [%]**
0.09454	0.05488	1.22441	97.65842	98.51403

Table 4.12: Performance metrics of the best performing LSTM architecture from the second batch on the test dataset

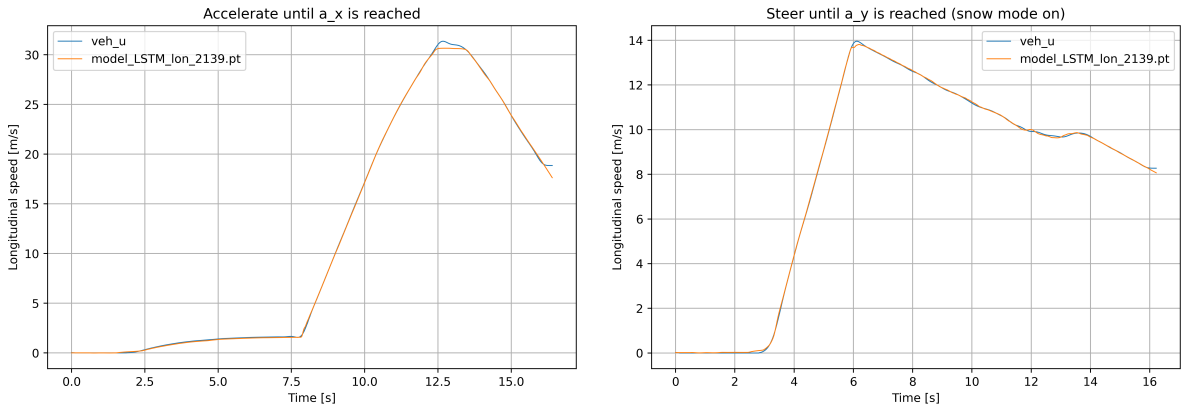


Figure 4.8: Predictions of the best performing LSTM architecture on the validation dataset after finetuning (left: worst case, right: best case)

It is interesting to note that the best performing architecture after the finetuning was the same as the best performing architecture from the first batch. This indicates that the initial hyperparameter ranges were well chosen and that the model was already close to optimal performance. The performance metrics remained unchanged, with an RMSE of 0.11949 m/s, MAE of 0.07529 m/s, and PWT 0.3 of 96.60422 %.

4.2.3 Gated Recurrent Unit - GRU

Theoretical background

The Gated Recurrent Unit (GRU) network also builds on the structure of the original RNN-s but introduces a simplified architecture compared to the LSTM network. The GRU uses similar gating mechanisms to control the flow of the input and the state information, but omits the separate cell state, therefore making it a lighter alternative. The structure of a GRU network unfolded in time can be seen on figure 4.9, where $\mathbf{x}_t$ is the input vector, $\mathbf{h}_t$ is the hidden state vector and $\mathbf{y}_t$ is the output vector at time step t.

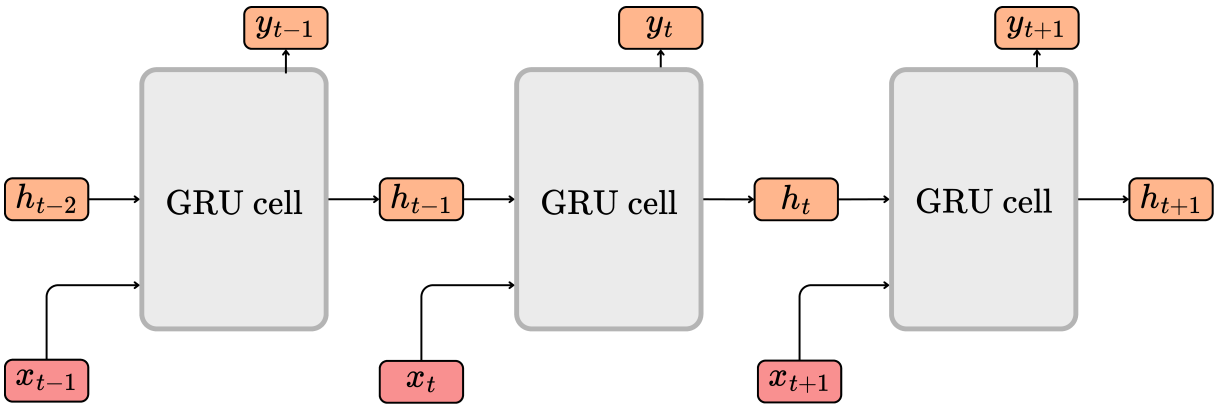


Figure 4.9: GRU sequence unfolded in time

The dimensions of the previously mentioned vectors are the following:

- Input vector: $\mathbf{x}_t \in \mathbb{R}^{d_x}$, where d_x is the size of the input vector.
- Hidden state: $\mathbf{h}_t \in \mathbb{R}^{d_h}$, where d_h is the number of hidden units in the GRU layer.
- Output vector: $\mathbf{y}_t \in \mathbb{R}^{d_y}$, where d_y is the size of the output vector.

Batch size B is not included here for simplicity the same way as before, but when considering training with batches of sequences, the dimensions would be: $\mathbf{x}_t \in \mathbb{R}^{B \times d_x}$, $\mathbf{h}_t \in \mathbb{R}^{B \times d_h}$, $\mathbf{y}_t \in \mathbb{R}^{B \times d_y}$.

The same way as with the LSTM cell, the white-box representation of the GRU cell can be seen on figure 4.10, where the weight matrices and the bias vectors are also presented. The dimensions of these vectors are the following:

- Weight matrices for the input vector: $\mathbf{W}_{xr}, \mathbf{W}_{xz}, \mathbf{W}_{xh} \in \mathbb{R}^{d_x \times d_h}$
- Weight matrices for the state vectors: $\mathbf{W}_{hr}, \mathbf{W}_{hz}, \mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$
- Weight matrix for the output vector: $\mathbf{W}_{hy} \in \mathbb{R}^{d_h \times d_y}$
- Bias vectors for the operation: $\mathbf{b}_r, \mathbf{b}_z, \mathbf{b}_h, \mathbf{b}_y \in \mathbb{R}^{d_h}, \mathbf{b}_y \in \mathbb{R}^{d_y}$

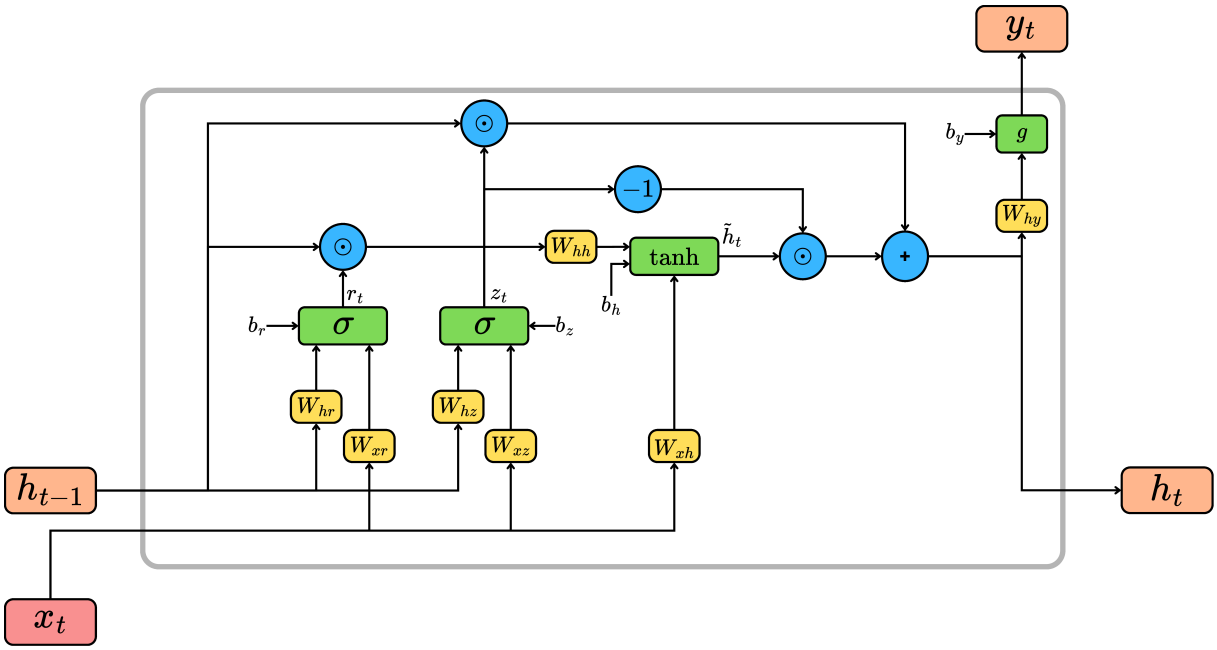


Figure 4.10: GRU unit with the gating mechanisms

The gate vectors that are within the GRU cell are the reset gate ($\mathbf{r}_t$) and the update gate ($\mathbf{z}_t$). These are mainly functions that use either sigmoid or tanh activation functions. Furthermore the cell contains operations to determine the candidate hidden state ($\tilde{\mathbf{h}}_t$), the new hidden state ($\mathbf{h}_t$) and the output ($\mathbf{y}_t$). All the gates and operations can be described with the following equations:

1. Reset gate 4.16: The reset gate determines how much of the previous hidden state ($\mathbf{h}_{t-1}$) shall be forgotten when calculating the candidate hidden state.

$$\mathbf{r}_t = \sigma(\mathbf{W}_{xr} \cdot \mathbf{x}_t + \mathbf{W}_{hr} \cdot \mathbf{h}_{t-1} + \mathbf{b}_r) \tag{4.16}$$

2. Update gate 4.17: The update gate decides how much of the previous hidden state shall be carried forward to the new hidden state.

$$\mathbf{z}_t = \sigma(\mathbf{W}_{xz} \cdot \mathbf{x}_t + \mathbf{W}_{hz} \cdot \mathbf{h}_{t-1} + \mathbf{b}_z) \tag{4.17}$$

3. Candidate hidden state 4.18: This operation creates a vector of new candidate values ($\tilde{\mathbf{h}}_t$) that could be added to the hidden state. It uses a tanh activation function to ensure the values are between -1 and 1.

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_{xh} \cdot \mathbf{x}_t + \mathbf{W}_{hh} \cdot (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h) \tag{4.18}$$

4. New hidden state 4.19: The new hidden state ($\mathbf{h}_t$) is computed by combining the previous hidden state ($\mathbf{h}_{t-1}$) scaled by the update gate and the candidate hidden state ($\tilde{\mathbf{h}}_t$) scaled by one minus the update gate.

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \tag{4.19}$$

5. Output 4.20: The output vector ($\mathbf{y}_t$) is determined from the new hidden state and the bias vector ($\mathbf{b}_y$).

$$\mathbf{y}_t = g(\mathbf{W}_{hy} \cdot \mathbf{h}_t + \mathbf{b}_y) \tag{4.20}$$

The training process of the GRU network is similar to that of the LSTM network, involving (BPTT) to optimize the weight matrices and bias vectors to minimize the defined loss (L) over a dedicated training dataset.

Implementation and results

The implementation and training procedure of the described GRU network is identical to the one used for the RNN-s and LSTM-s. The same reduced input set (12 inputs) was used. A grid search was performed over the hyperparameters to find the best combination of parameters that yield the best architecture. The training was done in a similar manner as with the RNN-s and LSTM-s. For the first batch of grid search the hyperparameter ranges are the as in table 4.1.

The results obtained after testing the created models on the same unseen test dataset. The code using for training and validating can be found at `AI_training/3_code/training_`

`GRU.py`. The 3 best performing architectures can be seen in table 4.13. The results that can be found in table 4.14 show that the GRU models outperformed the RNN models, but were slightly worse than the LSTM models, indicating that the GRU architecture is better suited for capturing the temporal dependencies in the vehicle speed estimation task than RNN-s, but not as good as LSTM-s.

	Input ID	**Sequence Length**	**Hidden size**	**Number of layers**
1.	2	100	64	1
2.	2	100	192	2
3.	2	50	96	2

Table 4.13: Results of the best performing GRU architecture on the test dataset

	RMSE $[\frac{m}{s}]$	**MAE** $[\frac{m}{s}]$	**AMaxE** $[\frac{m}{s}]$	**PWT 0.3 [%]**	**PWT 0.4 [%]**
1.	0.13113	0.07793	1.16016	95.79582	97.22842
2.	0.13209	0.07542	1.17943	96.02108	97.03984
3.	0.13440	0.07439	1.34688	96.43437	97.41938

Table 4.14: Performance metrics of the best performing GRU architectures on the test dataset

It can be seen that the GRU has performed better than the best RNN model, but slightly lacks behind the best LSTM model. After evaluating these results another grid search was performed around the 3 best performing architectures to see if further improvements could be achieved by finetuning the hyperparameters. The hyperparameter ranges for the finetuning can be seen in table 4.15. Overall 72 new models were created and evaluated.

Around 1st best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	48, 64, 80
Number of layers	1, 2
Sequence length	90, 100, 110
Around 2nd best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	176, 192, 208
Number of layers	1, 2, 3
Sequence length	90, 100, 110
Around 3d best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	80, 96, 112
Number of layers	1, 2, 3
Sequence length	40, 50, 60

Table 4.15: Batch 2: Hyperparameter ranges for GRU architecture search

The best model obtained (4.15) from the second batch of grid search was evaluated on the unseen test dataset. The results of this best model can be seen in table 4.17. The best model's estimates for the validation dataset can be seen on figure 4.11.

	Input ID	**Sequence Length**	**Hidden size**	**Number of layers**
1.	2	90	208	3

Table 4.16: Results of the best performing GRU architecture on the test dataset

RMSE [$\frac{m}{s}$]	**MAE [$\frac{m}{s}$]**	**AMaxE [$\frac{m}{s}$]**	**PWT 0.3 [%]**	**PWT 0.4 [%]**
0.09454	0.05488	1.22441	97.65842	98.51403

Table 4.17: Performance metrics of the best performing GRU architecture from the second batch on the test dataset

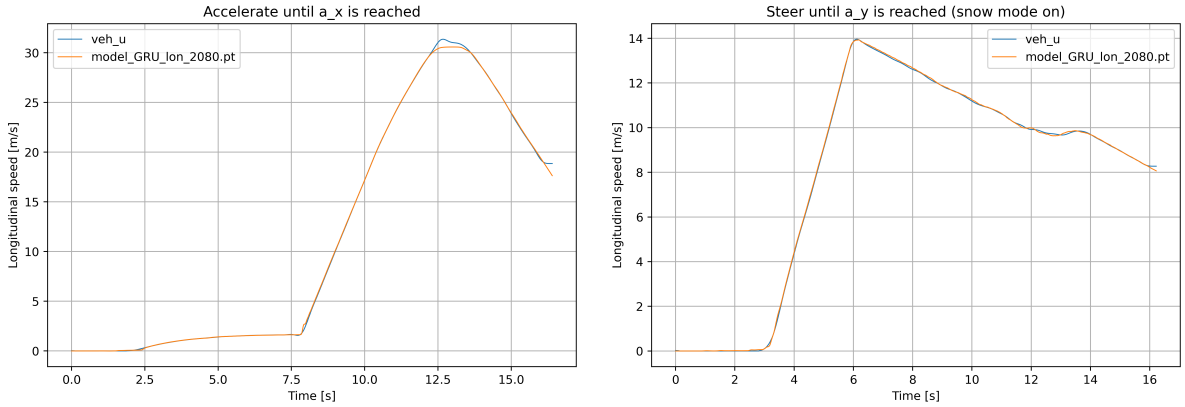


Figure 4.11: Predictions of the best performing GRU architecture on the validation dataset after finetuning (left: worst case, right: best case)

The best performing GRU architecture from the second batch achieved an RMSE of 0.09454 m/s, MAE of 0.05488 m/s, and PWT 0.3 of 97.65842 %. These results indicate the GRU network's capability falls between that of the RNN and LSTM networks for the vehicle speed estimation task.

4.3 Temporal Convolutional Networks - TCN

Theoretical background

The Temporal Convolutional Network (TCN) is a type of convolutional neural network (CNN) that is specifically designed for handling sequential data. Unlike RNN-s and their variants, TCN-s utilize convolutional layers to capture temporal dependencies in the data. The key features of TCN-s include causal convolutions, dilated convolutions, and residual connections. The described TCN architecture is based on the work of article [18].

The TCN produces the output sequence that has the same length as the input sequence. This is due to using a 1D fully convolutional network (FCN) architecture, meaning that each hidden layer is the same length as the input sequence. This property is fulfilled by using zero-padding at the beginning of the input sequence.

Causal convolutions ensure that the output at time step $\mathbf{y}_t$ is only dependent on the inputs from time steps t and earlier. Dilated convolutions allow the network to have a larger receptive field without increasing the number of parameters significantly. This is achieved by introducing gaps between the filter elements, enabling the network to capture long-term dependencies more effectively. The dilated causal convolutions, i.e the base of the TCN, can be seen on figure 4.12, where the dilation factor d determines the spacing between the filter elements and the kernel size k defines the size of the filter.

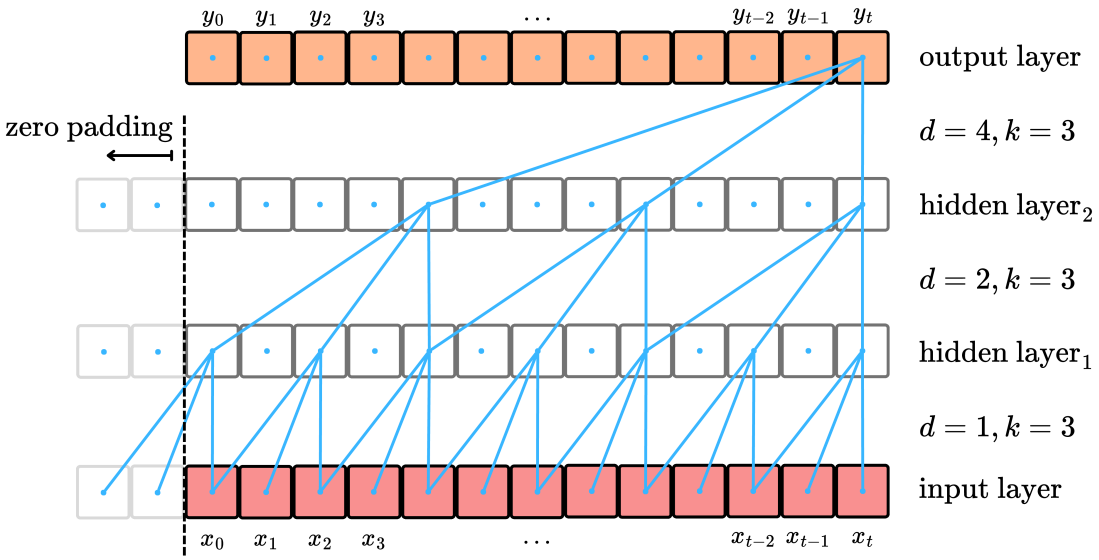


Figure 4.12: Dilated causal convolution

The dimensions of the vectors are the following:

- Input vector at time step t: $\mathbf{x}_t \in \mathbb{R}^{d_x}$, where d_x is the channel size of the input vector. Channel size refers to the number of features in the input vector at each time step.
- Output vector at time step t: $\mathbf{y}_t \in \mathbb{R}^{d_y}$, where d_y is the chanel size of the output vector.
- Sequence length: L, the total number of time steps in the input sequence.
- Bias vector: $\mathbf{b} \in \mathbb{R}^{d_y}$
- Weight tensor: $\mathbf{W} \in \mathbb{R}^{d_y \times d_x \times k}$, where k is the kernel size of the convolutional filter.
- Input tensor: $\mathbf{X}_t = [\mathbf{x}_0, \mathbf{x}_1, ..., \mathbf{x}_t]^T \in \mathbb{R}^{L \times d_x}$
- Output tensor: $\mathbf{Y}_t = [\mathbf{y}_0, \mathbf{y}_1, ..., \mathbf{y}_t]^T \in \mathbb{R}^{L \times d_y}$

Based on the dilation and the kernel size between two layers, the equation for determining the output can the formulated as (if there are no hidden layers):

$$\mathbf{y}_t = \sum_{i=0}^{k-1} \mathbf{w}_i \cdot \mathbf{x}_{t-d \cdot i} + \mathbf{b}, \tag{4.21}$$

where $\mathbf{y}_t$ is the output vector at time step t, $\mathbf{w}_i$ are the weights of the filter, $\mathbf{b}$ is the bias term, k is the kernel size, and d is the dilation factor between the two layers.

The full equation considering the input and output tensors between two layers can be formulated as:

$$\mathbf{Y}_{j,t} = \sum_{i=1}^{d_x} \sum_{m=0}^{k-1} \mathbf{W}_{j,i,m} \cdot \mathbf{X}_{i,t-d \cdot m} + \mathbf{b}_j, \tag{4.22}$$

where $\mathbf{Y}_{j,t}$ is the jth vector element of the output tensor at time step t, $\mathbf{W}_{i,j,m}$ contains the weights of the filter connecting input channel i to output channel j at position m (in the kernel), and $\mathbf{b}_j$ is the bias term for output channel j. Iterating this equation all the vector elements of tensor $\mathbf{Y}$ can be determined based on the elements of the input tensor $\mathbf{X}$.

This can be generalized for two arbitrary layers that are connected through a 1D dilated causal convolution as:

$$\mathbf{H}_{j,t}^{(l)} = \sum_{i=1}^{d_h^{(l-1)}} \sum_{m=0}^{k-1} \mathbf{W}_{j,i,m}^{(l)} \cdot \mathbf{H}_{i,t-d^{(l)} \cdot m}^{(l-1)} + \mathbf{b}_j^{(l)} = \text{Conv1D}(\mathbf{H}^{(l-1)}), \tag{4.23}$$

where $\mathbf{H}_{j,t}^{(l)}$ is the jth vector element in time step t of the output tensor of layer l, $d_h^{(l-1)}$ is the number of channels in the tensor of layer $l-1$, $\mathbf{W}_{j,i,m}^{(l)}$ contains the weights of the filter connecting input channel i to output channel j at position m and $\mathbf{b}_j^{(l)}$ is the jth bias for layer l.

After each convolutional layer, a non-linear activation function is applied (e.g., ReLU or tanh) to introduce non-linearity into the model. Moreover, a dropout and a normalization layer is often added after the activation function to prevent overfitting by randomly setting a fraction of the input units to zero during training. Finally a residual connection is added to the output of the convolutional block. This set of operations can be called a TCN residual block. Here it is important to ensure that the input and the output have the same dimensions to ensure the element-wise addition. These steps can be summarized as:

$$\mathbf{H}^{(l)} = \text{Dropout}(\text{Activation}(\text{Conv1D}(\mathbf{H}^{(l-1)}))) + \text{Conv1x1}(\mathbf{H}^{(l-1)}) \tag{4.24}$$

These steps build up a single TCN residual block that can be seen on figure 4.13. By stacking multiple such blocks on top of each other, a deep TCN architecture can be created that is capable of capturing complex temporal dependencies in the input data.

The amount of inputs that the TCN can consider when making a prediction is determined by the receptive field of the network. The receptive field (R) can be calculated based on the kernel size (k), the number of residual blocks (L), the number of convolutional layers per block (B) and the dilation schedule ($d_1, d_2, ...d_n$) as:

$$R = 1 + (k-1) \cdot B \cdot \sum_{i=1}^{L} d_i \tag{4.25}$$

During the training process, the objective is the same as with the RNN models - to optimize the weight matrices and bias vectors to minimize the defined loss (L) over a dedicated training dataset. The training is porformed just as with a FNN, using back-propagation to calculate the gradients of the loss with respect to the weights and biases,

and updating them using an optimizer such as Adam or SGD.

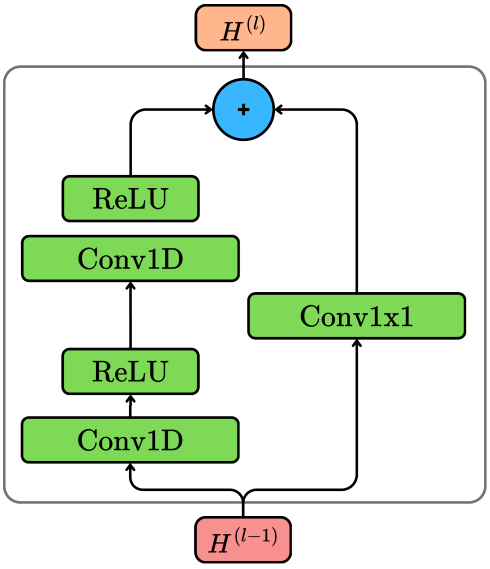


Figure 4.13: TCN residual block with two dilated causal convolutional layers

Implementation and results

The implementation and training procedure of the TCN network is similar to the on used in the RNN based models. The same reduced input set was used which consists of 12 inputs. A grid search was performed over the parameter space. The parameters that were varied during the grid search are the following: kernel size (k), dilation schedule between the layers ($d_1, d_2, ...d_n$), channels per layer (d_h), number of residual blocks (L) sequence length (L_{seq}) and convolutions per residual block (B). The value ranges for the first batch of the grid search can be seen in table 4.18.

Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Kernel size (k)	3, 4, 5
Dilation schedule (d_{sched})	[1, 2, 4, 8], [1, 2, 4, 8, 16], [1, 2, 4, 8, 16, 32]
Channels per layer (d_h)	64
Number of residual blocks (L)	4, 5, 6
Number of layers per convolution (L_c)	$L_c = L$
Convolutions per residual block (B)	2
Sequence length (L_{seq})	50, 100, 150, 200

Table 4.18: Batch 1: Hyperparameter ranges for TCN architecture search

After the training and evaluation of the models from the first batch of grid search, the 2 best performing architecture was selected based on the same principle as before. The results of these best models can be seen in table 4.20. Alltogether 25 models were created

and evaluated during the first batch of grid search. The code using for training can be found at `AI_training/3_code/training_TCN.py`.

	k	d_{sched}	d_h	L	L_{seq}	B
1.	4	[1, 2, 4, 8]	64	4	50	2
2.	4	[1, 2, 4, 8, 16]	64	5	150	2
3.	4	[1, 2, 4, 8, 16, 32]	64	6	150	2

Table 4.19: Results of the best performing TCN architecture on the test dataset

RMSE $[\frac{m}{s}]$	**MAE** $[\frac{m}{s}]$	**AMaxE** $[\frac{m}{s}]$	**PWT 0.3 [%]**	**PWT 0.4 [%]**
0.10200	0.06605	1.16851	98.30497	99.17077
0.11550	0.07546	1.16850	97.56255	99.05121
0.11713	0.07767	1.18234	98.04922	99.03550

Table 4.20: Performance metrics of the best performing TCN architectures from the first batch on the test dataset

After evaluating these results another grid search was performed around the 3 best performing architectures to see if further improvements could be achieved by finetuning the hyperparameters. The hyperparameter ranges for the finetuning can be seen in table 4.21.

Around 1st best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Kernel size (k)	3, 4, 5
Dilation schedule (d_{sched})	[1, 2, 4, 8], [1, 2, 4, 8, 16], [1, 2, 4, 8, 16, 32]
Channels per layer (d_h)	64
Convolutions per residual block (B)	2, 3
Number of residual blocks (L)	3, 4, 5
Sequence length (L_{seq})	30, 40, 50, 60, 70
Around 2nd best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Kernel size (k)	3, 4, 5
Dilation schedule (d_{sched})	[1, 2, 4, 8], [1, 2, 4, 8, 16], [1, 2, 4, 8, 16, 32]
Channels per layer (d_h)	64
Convolutions per residual block (B)	2, 3
Number of residual blocks (L)	4, 5, 6
Sequence length (L_{seq})	130, 140, 150, 160, 170
Around 3d best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Kernel size (k)	3, 4, 5
Dilation schedule (d_{sched})	[1, 2, 4, 8], [1, 2, 4, 8, 16], [1, 2, 4, 8, 16, 32]
Channels per layer (d_h)	64
Convolutions per residual block (B)	2, 3
Number of residual blocks (L)	5, 6, 7
Sequence length (L_{seq})	130, 140, 150, 160, 170

Table 4.21: Batch 2: Hyperparameter ranges for TCN architecture search

The best model obtained after evaluating the second batch was still the same model as before (4.21). The results of this best model can be seen in table 4.22. The best model's estimates for the validation dataset can be seen on figure 4.14.

RMSE [$\frac{m}{s}$]	**MAE [$\frac{m}{s}$]**	**AMaxE [$\frac{m}{s}$]**	**PWT 0.3 [%]**	**PWT 0.4 [%]**
0.07513	0.04274	1.19855	99.01150	99.38940

Table 4.22: Performance metrics of the best performing TCN architecture from the second batch on the test dataset

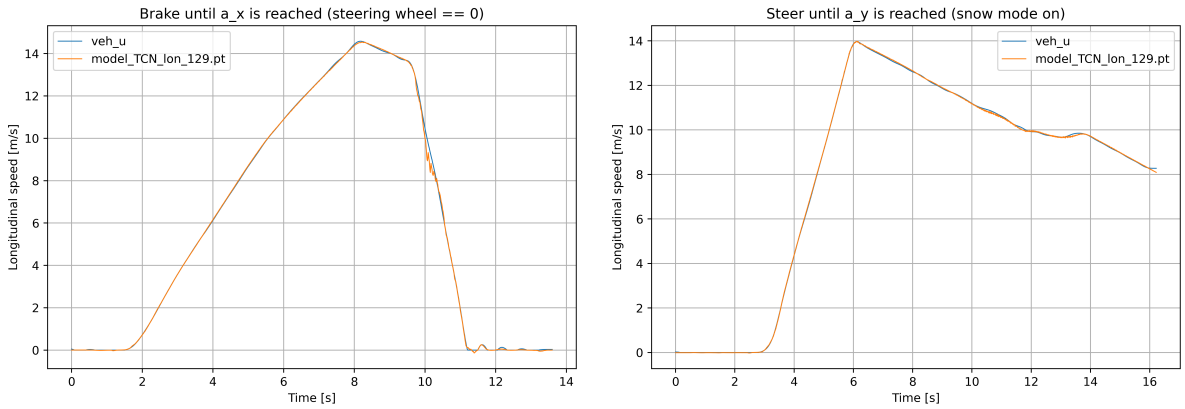


Figure 4.14: Predictions of the best performing TCN architecture on the validation dataset after finetuning (left: worst case, right: best case)

It can be seen that the TCN architecture outperforms all the previously discussed RNN based architectures, achieving an RMSE of 0.07513 m/s, MAE of 0.04274 m/s, and PWT 0.3 of 99.01150 %. These results highlight the effectiveness of TCN-s in capturing temporal dependencies in sequential data for the vehicle speed estimation task.

4.4 Transformer

Theoretical background

The Transformer architecture is a deep learning model that has revolutionized the field of natural language processing (NLP) and has also found applications in other sequential data tasks. Unlike traditional RNN-s and TCN-s where the models rely on recurrence or convolution, the transformer relies entirely on self-attention mechanisms to capture dependencies in the input data, allowing for parallel processing of sequences and improved handling of long-range dependencies. An other important aspect of the transformer architecture is that it does not take the sequence data in a step-by-step manner, but rather processes the entire sequence at once, making it different from the previously discussed models.

The general architecture of the transformer model can be divided into two main components: the encoder and the decoder. The encoder is responsible for processing the input sequence and generating a set of continuous representations, while the decoder takes these representations and generates the output sequence. The original article published by Google [19] describes the working principle of the full transformer architecture including both the encoder and decoder parts. However, for tasks such as vehicle speed estimation, only the encoder part is enough, as there is no need to generate a sequence output.

The steps of the transformer encoder can be seen on 4.15 and summarized as follows:

1. Input embedding: The input sequence $X_t \in \mathbb{R}^{L \times d_x}$ at time step t, where L is the sequence length and d_x is the size of the input vector, is first converted into a continuous representation using an embedding layer. This layer maps each input to a high-dimensional vector space d_m.

$$E_t = \text{Embedding}(X_t) = W_e \cdot X_t + b_e, \tag{4.26}$$

where $E_t \in \mathbb{R}^{L \times d_m}$ is the embedded input vector at time step t, $W_e \in \mathbb{R}^{d_m}$ is the embedding weight matrix, and $b_e \in \mathbb{R}^{d_m}$ is the bias term.

2. Positional encoding: Since the transformer architecture does not rely on the order of the sequence, positional encodings are added to the embedded input vectors to provide information about the position of each element in the sequence. This can be done using sinusoidal functions or learned positional embeddings. The original transformer paper [19] proposes the following sinusoidal positional encoding:

$$\text{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_m}}\right), \tag{4.27}$$

$$\text{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_m}}\right), \tag{4.28}$$

where pos is the position in the sequence and i is the dimension index. After the positional encodings are calculated, they are added to the embedded input vectors:

$$X'_t = E_t + PE, \tag{4.29}$$

where X'_t is the input embedded vector with added positional encodings.

3. Multi-head self-attention: This mechanism allows the model to focus on different parts of the input sequence simultaneously. The input is transformed into three matrices: queries (Q), keys (K), and values (V) using learned weight matrices (W_Q, W_K and W_V). The attention scores are calculated using the following equation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{4.30}$$

where d_k is the dimension of the key vectors. The term multi-head attention implies that the process is repeated multiple times (heads) with different learned weight matrices, allowing the model to capture various aspects of the input sequence. The outputs of each head are then concatenated and transformed using another learned weight matrix (W_O):

$$\text{head}_i = \text{Attention}(X'_t W_{Q_i}, X'_t W_{K_i}, X'_t W_{V_i}), \tag{4.31}$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, ..., \text{head}_h)W_O, \tag{4.32}$$

where h is the number of attention heads.

4. Add and Normalization: After the multi-head self-attention layer, a residual connection is added that is followed by layer normalization. This helps in stabilizing the training process and allows for better gradient flow.

$$X_{attn} = \text{LayerNorm}(X'_t + \text{MultiHead}(X'_t)), \tag{4.33}$$

where $X_{attn} \in \mathbb{R}^{L \times d_m}$ is the output of the multi-head self-attention layer at time step t.

5. Feed-forward neural network: After the multi-head self-attention layer, a position-wise feed-forward neural network (FFN) is applied to each position in the sequence independently. This FFN consists of two linear transformations with an activation function (eg. ReLU):

$$\text{FFN}(X_{attn}) = W_2\text{act}(W_1 X_{attn} + b_1) + b_2, \tag{4.34}$$

where $W_1 \in \mathbb{R}^{d_m \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d_m}$ are the weight matrices, and $b_1 \in \mathbb{R}^{d_{ff}}$, $b_2 \in \mathbb{R}^{d_m}$ are the bias terms. d_{ff} is the dimension of the feed-forward layer.

6. Add and Normalization: Similar to the previous step, a residual connection is added followed by layer normalization after the feed-forward neural network.

$$Y_t = \text{LayerNorm}(X_{attn} + \text{FFN}(X_{attn})), \tag{4.35}$$

where $Y_t \in \mathbb{R}^{L \times d_m}$ is the output of the transformer encoder at time step t.

7. Regression head: Finally, a regression head is added on top of the transformer encoder to map the output representations to the desired output size ie. the vehicle speed. This can be done using a simple feed-forward layer:

$$\hat{y}_t = W_{out} \cdot Y_t + b_{out}, \tag{4.36}$$

where $\hat{y}_t \in \mathbb{R}^{L \times d_y}$ is the predicted output vector at time step t, $W_{out} \in \mathbb{R}^{d_m \times d_y}$ is the output weight matrix, and $b_{out} \in \mathbb{R}^{d_y}$ is the bias term. From here the last element of the output vector can be taken as the estimated vehicle speed at the current time step.

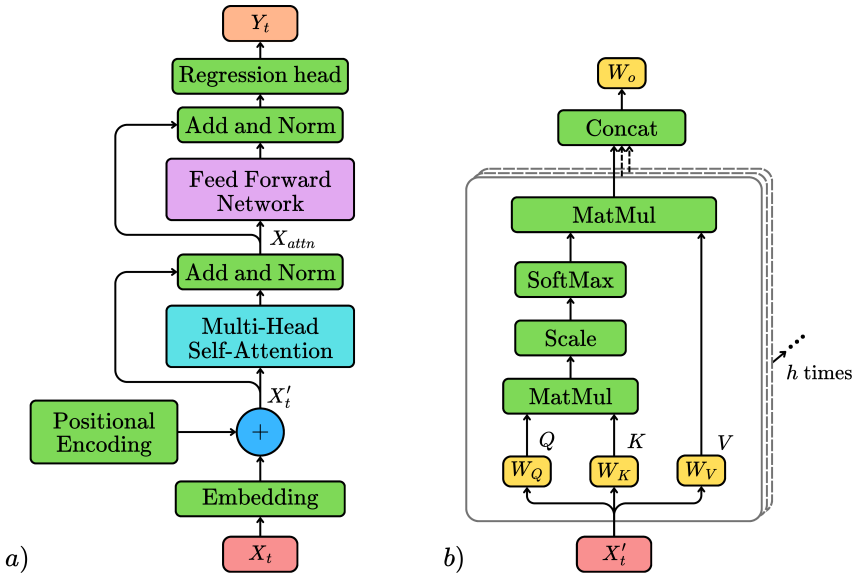


Figure 4.15: a) Transfomer encoder block with regression head b) Multi-head self-attention mechanism

Implementation and results

The implementation and training procedure of the transformer encoder is similar to the one used in the previous models. The same reduced input set was used which consists of 12 inputs. A grid search was performed over the parameter space. The parameters that were varied during the grid search are the following: model dimension (d_m), feed-forward dimension (d_{ff}), number of layers (L), number of attention heads (n_{head}) and sequence length (L_{seq}). The value ranges for the first batch of the grid search can be seen in table 4.23. Overall 64 models were created and evaluated during the first batch of grid search.

Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Model dimension (d_m)	32, 64
Feed-forward dimension (d_{ff})	64, 128, 96
Number of layers (L)	2, 3
Number of attention heads (n_{head})	2, 4
Sequence length (L_{seq})	50, 100, 150, 200

Table 4.23: Batch 1: Hyperparameter ranges for transformer architecture search

To note, only the first two best models will be investigated onwards, as the training sequence is much more computationally intense as for the prevous models. The results of the best 2 models obtained from the first batch (4.24) of grid search can be seen in table 4.25. The code using for training and validating can be found at `AI_training/3_code/training_transformer.py`.

	d_m	d_{ff}	L	n_{head}	L_{seq}
1.	32	96	2	2	50
2.	32	3	96	4	100

Table 4.24: Batch 1: Results of the best performing transformer architecture on the test dataset

RMSE [$\frac{m}{s}$]	**MAE [$\frac{m}{s}$]**	**AMaxE [$\frac{m}{s}$]**	**PWT 0.3 [%]**	**PWT 0.4 [%]**
0.23937	0.16788	2.46572	83.88168	93.43833
0.24577	0.16074	2.62185	90.82006	93.52729

Table 4.25: Batch 1: Performance metrics of the best performing transformer architectures from the first batch on the test dataset

It can be seen that the results obtained from the trained transfromer models fall behind the previously developed models. To continue, another grid search was performed around the 2 best performing architectures to see if further improvements could be achieved by finetuning the hyperparameters. The hyperparameter ranges for the finetuning can be seen in table 4.26.

Around 1st best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Model dimension (d_m)	16, 32, 64
Feed-forward dimension (d_{ff})	32, 48, 96, 128, 192, 256
Number of layers (L)	1, 2, 3
Number of attention heads (n_{head})	2, 4
Sequence length (L_{seq})	30, 40, 50, 60, 70
Around 2nd best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Model dimension (d_m)	16, 32, 64
Feed-forward dimension (d_{ff})	32, 48, 96, 128, 192, 256
Number of layers (L)	2, 3, 4
Number of attention heads (n_{head})	2, 4, 8
Sequence length (L_{seq})	80, 90, 100, 110, 120

Table 4.26: Batch 2: Hyperparameter ranges for transformer architecture search

During the second batch overall 1214 new models were created and trained. The best model obtained from the second batch of grid search was evaluated on the unseen test dataset. The parameters and the results of this best model can be seen in table 4.26 and 4.28. The best model's estimates for the validation dataset can be seen on figure 4.16.

	d_m	d_{ff}	L	n_{head}	L_{seq}
1.	16	32	2	2	110

Table 4.27: Batch 2: Results of the best performing transformer architecture on the test dataset

RMSE $[\frac{m}{s}]$	**MAE** $[\frac{m}{s}]$	**AMaxE** $[\frac{m}{s}]$	**PWT 0.3 [%]**	**PWT 0.4 [%]**
0.17445	0.12758	1.58004	92.12807	96.93576

Table 4.28: Performance metrics of the best performing transformer architecture from the second batch on the test dataset

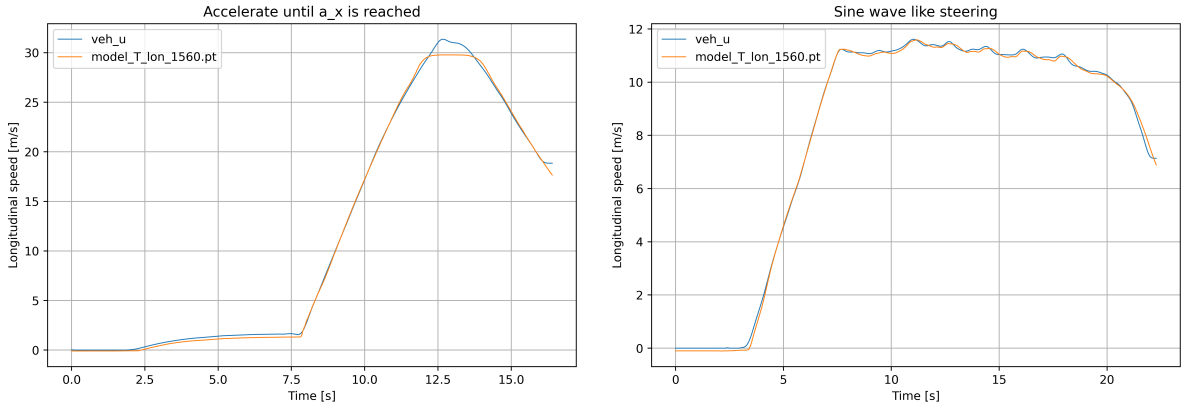


Figure 4.16: Predictions of the best performing transformer architecture on the validation dataset after finetuning (left: worst case, right: best case)

The best obtained transformer model achieved an RMSE of 0.17445 m/s, MAE of 0.12758 m/s, and PWT 0.3 of 92.12807 %. These results indicate that the transformer architecture, in this case, does not perform as well as the previously discussed RNN and TCN based architectures for the vehicle speed estimation task. This could be due to the fact that transformers are typically more effective in handling long-range dependencies in sequences and are used extensively in NLP tasks. However, for the specific task of vehicle speed estimation, other architectures like TCN proved to be more suitable.

4.5 Summary and comparison of the sequential models

Throughout the chapter 5 different AI architectures were presented. These included RNN-s (vanilla RNN, LSTM, GRU), TCN-s and transformer encoders. All the architectures were trained and evaluated on the same dataset using the same input features. The performance metrics of the best performing models from each architecture can be seen in table 4.29.

Model	**RMSE**	**MAE**	**AMaxE**	**PWT 0.3**	**PWT 0.4**
Vanilla RNN	0.12640	0.06590	1.49319	96.91017	97.52393
LSTM	0.09454	0.05488	1.22441	97.65842	98.51403
GRU	0.10588	0.05547	1.32567	96.78761	97.78252
TCN	**0.07513**	**0.04274**	**1.19855**	**99.01150**	**99.38940**
Transformer	0.17445	0.12758	1.58004	92.12807	96.93576

Table 4.29: Comparison of the best performing AI models on the test dataset

The results indicate that the TCN architecture outperforms all the other models, achieving the lowest RMSE and MAE values, as well as the highest PWT 0.3 and PWT 0.4 percentages. This suggests that TCN-s are particularly effective at capturing temporal dependencies in the input data for vehicle speed estimation tasks.

Chapter 5

AI aided estimation

In the previous chapter the direct estimation of the vehicle speed with different AI models was discussed. In this chapter the focus will be shifted towards combining conventional estimation methods with AI algorithms to improve the overall performance of the estimator. First a simple wheel speed based vehicle speed estimation method will be presented. Then a model based estimator will be discussed that utilizes the linear tire model to estimate the vehicle speed from the tire forces. Furthermore an Extended Kalman Filter (EKF) based estimator will be developed that fuses the previously derived model based estimator as the state transition function and the wheel speed based estimator as the measurement function. Finally, the best performing AI model from the previous chapter will be combined with the EKF based estimator to improve its performance.

An important notice is that all the newly presented estimation methods are based on the simplification that the vehicle is a 2D rigid body moving on a flat surface. Therefore the vertical dynamics of the vehicle are neglected and the roll and pitch anlges are considered to be zero.

5.1 Best wheel based speed estimation

Governing equations

The best wheel based speed estimation method is a simple yet effective approach to estimate the vehicle speed using the rotational speeds of the individual wheels. The main idea behind it is to calculate the vehicle speed from the rotational speeds of the wheels, transform these speeds into the Center of Gravity (CoG) of the vehicle and then take the best estimate from the four wheels.

The best estimate is determined based on the vehicle's longitudinal acceleration. If the vehicle is accelerating ($a_x > 0$), the wheel with the minimum angular velocity is chosen as the best estimate, as it is less likely to be affected by wheel slip. With the same approach, if the vehicle is decelerating ($a_x < 0$), the wheel with the maximum angular velocity is

selected, as it is less likely to be affected by braking slip.

The sensor measurements needed for this estimation method are the four wheel speeds (ω^{FL}, ω^{FR}, ω^{RL}, ω^{RR}), the longitudinal acceleration (a_x) of the vehicle, the yaw rate ($\dot{\psi}$) of the vehicle and the road wheel angles (δ^{FL}, δ^{FR}, δ^{RL}, δ^{RR}). These are all available from the vehicle's Inertial Measurement Unit (IMU), wheel speed sensors and the electric power steering system (EPS). Furthermore the vehicle's geometric parameters such as the average effective wheel radius of the wheels (R) and the positional vectors pointing from the CoG to the wheels ($\mathbf{r}^{FL}$, $\mathbf{r}^{FR}$, $\mathbf{r}^{RL}$, $\mathbf{r}^{RR}$) are also required.

The calculation steps of the described estimator are the following. Calculation of the velocity at each wheel (v_x^i) from the wheel speeds (ω^i)

$$v_{w,x}^i = \omega^i \cdot R, \tag{5.1}$$

where $i \in \{FL, FR, RL, RR\}$. This way the longitudinal velocity at each wheel is obtained,

$$\mathbf{v}_w^i = [v_{w,x}^i, 0, 0]^T. \tag{5.2}$$

It needs to mentioned here that this is just an approximation, as the lateral velocity component at the wheel is neglected. The next step is to express the determined wheel velocity vectors in the frame that is aligned with the frame of the body and is centered at the wheels. This is done by applying a rotation matrix (T_{δ_i}) based on the road wheel angles (δ^i) of the respective wheels. The rotation matrix is the can be written as

$$\mathbf{T}_{\delta^i} = \begin{bmatrix} \cos(\delta^i) & -\sin(\delta^i) & 0 \\ \sin(\delta^i) & \cos(\delta^i) & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{5.3}$$

After rotating the wheel velocities to the appropriate frame, the velocity at the CoG of the vehicle can be calculated by using the yaw rate ($mathbf\eta = [0, 0, \dot{\psi}]^T$) of the vehicle and the positional vectors pointing from the wheels to the CoG (r_i). The velocity at the CoG can be calculated as

$$\mathbf{v}_{cog}^i = \mathbf{T}_{\delta^i} \cdot \mathbf{v}_w^i + \eta \times \mathbf{r}^i, \tag{5.4}$$

where $\mathbf{v}_{cog}^i$ are the velocity vectors at the CoG calculated from the four different wheels. After calculating the CoG velocities from each wheel, the best estimate can be selected based on the longitudinal acceleration (a_x) of the vehicle. The final vehicle speed estimate ($\mathbf{v}_{est}$) can be written as

$$\mathbf{v}_{est} = \begin{cases} \min(\mathbf{v}_{cog}^{FR}, \mathbf{v}_{cog}^{FL}, \mathbf{v}_{cog}^{RR}, \mathbf{v}_{cog}^{RL}) & \text{if } a_x \geq 0 \\ \max(\mathbf{v}_{cog}^{FR}, \mathbf{v}_{cog}^{FL}, \mathbf{v}_{cog}^{RR}, \mathbf{v}_{cog}^{RL}) & \text{if } a_x < 0 \end{cases}. \tag{5.5}$$

Implementation and results

The best wheel based speed estimator was implemented in MATLAB Simulink envionment. The function that realizes the described calculation can be found at `Conventional_estimators/Best_wheel_based_speed_estimator.m`. The estimator was tested on the same test dataset as the AI based direct estimators from the previous chapter. The results of the evaluation can be seen in table 5.1.

The parameters of the vehicle that were used for the estimation are the following: effective wheel radius R, positional vectors from COG to wheels: $\mathbf{r}^{FL}$, $\mathbf{r}^{FR}$, $\mathbf{r}^{RL}$, $\mathbf{r}^{RR}$. The values of the parameters can be found in table 5.2.

RMSE $[\frac{m}{s}]$	**MAE** $[\frac{m}{s}]$	**AMaxE** $[\frac{m}{s}]$	**PWT 0.3 [%]**	**PWT 0.4 [%]**
0.69604	0.36960	6.41560	69.24063	73.63348

Table 5.1: Performance metrics of the best wheel based speed estimator on the test dataset

It can be seen that the best wheel based speed estimator provides a decent performance on the test dataset. It is important to note that this esimation method works well under straight line driving conditions with low longitudinal and lateral slip. However, during maneuvering scenarios where the steering wheel angle is not zero or braking is apllied the accuracy falls behind drastically due to the simplifications made in the calculation of the wheel velocities. This can be observed in figure 5.1, where the estimator performs well during straight line driving but shows significant errors during cornering maneuvers.

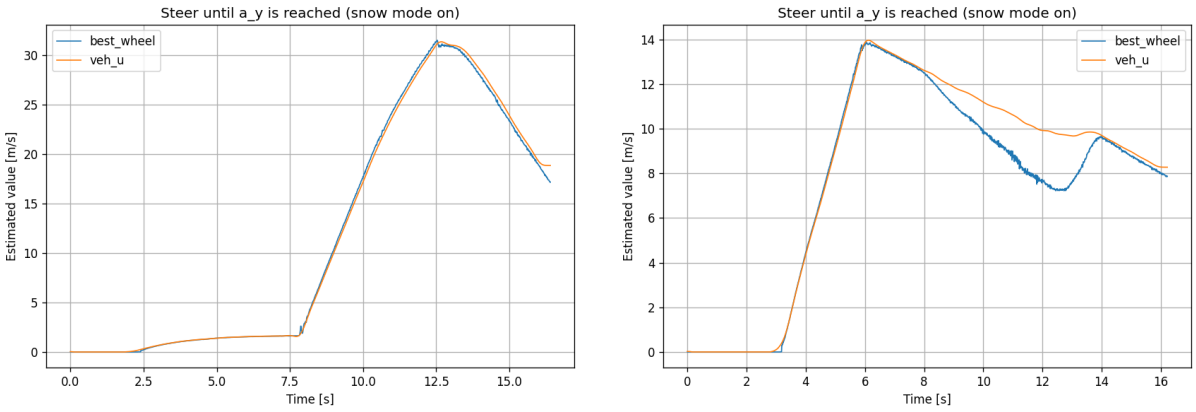


Figure 5.1: Low longitudinal and lateral slip in straight line driving (left) and high lateral slip during cornering (right)

5.2 Model based speed estimation

Governing equations

In this section the intention is to create a function that can determine the vehicle speeds time derivatives based on the available sensor measurements and the vehicle parameters. The developed model based estimator will be later used as the state transition function in the EKF based estimator.

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}), \tag{5.6}$$

where $\mathbf{x} = [v_x, v_y, \dot{\psi}_z]^T$ is the chosen state vector containing the longitudinal speed, the lateral speed and the yaw rate of the vehicle. The input vector is defined as:

$$\mathbf{u} = [\omega^i, \delta^i, m, I_z, c^j_{lon}, c^j_{lat}, R, \mathbf{r}^i, dt, \epsilon]^T, \tag{5.7}$$

where ω^i are the four wheel speeds, δ^i are the four road wheel angles, m is the mass of the vehicle, I_z is the yaw moment of inertia of the vehicle, c^j_{lon} and c^j_{lat} are the longitudinal and lateral tire stiffness coefficients for the front and rear wheels, R is the effective wheel radius, r^i are the positional vectors pointing from the CoG to the wheels, dt is the sampling time and ϵ is a small constatn that will be used at the longitudinal slip calculation. The indices $i \in \{FL, FR, RL, RR\}$ and $j \in \{F, R\}$.

The first step of the state transition function is to determine the velocity vectors at each wheel ($\mathbf{v}^i_w$). For velocity at each wheel needs to be calulated. This can be done the same way as in the previous section:

$$\mathbf{v}^i_w = \mathbf{T}^{-1}_{\delta^i} \cdot (\mathbf{v}_{cog} - \eta \times \mathbf{r}^i), \tag{5.8}$$

where $\mathbf{v}_{cog} = [v_x, v_y, 0]^T$, $\eta = [0, 0, \dot{\psi}]^T$, $\mathbf{T}_{\delta^i}$ is the rotation matrix based on the road wheel angles (δ^i) of the respective wheels (same as in equation 5.3). The next step is to calculate the longitudinal slip ratios (κ^i) and lateral side slip angles (α^i) at each wheel. The longitudinal slip can be calculated as

$$\kappa^i = \frac{\omega^i \cdot R - v^i_{w,x}}{\max(|v^i_{w,x}|, \epsilon)}, \tag{5.9}$$

where ϵ is a small constant to avoid division by zero. The sideslip angle at each wheel can be determined as

$$\alpha^i = \arctan\left(\frac{-v^i_{w,y}}{\max(|v^i_{w,x}|, \epsilon)}\right). \tag{5.10}$$

From here the tire forces at each wheel will be determined using the linear tire model,

that was presented in [20]. The longitudinal tire forces (F_x^i) can be calculated as

$$F_x^i = c_{lon}^j \cdot \kappa^i, \tag{5.11}$$

where $j \in \{F, R\}$ depending on whether the wheel is a front or rear wheel. The lateral tire forces (F_y^i) can be determined as

$$F_y^i = c_{lat}^j \cdot \alpha^i. \tag{5.12}$$

With this the tire force vectors at each wheel can be expressed as

$$\mathbf{F}_{tire}^i = [F_x^i, F_y^i, 0]^T. \tag{5.13}$$

After determining the tire forces at each wheel, the total force acting on the vehicle can be calculated by transforming the tire forces to the vehicle's body frame and summing them up:

$$\mathbf{F}_{total} = \sum_i \mathbf{T}_{-\delta^i}^T \cdot \mathbf{F}_{tire}^i, \tag{5.14}$$

From the transformed tire forces at the wheels the total moment acting on the vehicle around the vertical axis can be calculated as it is presented in [20]:

$$M_z = \sum_i (\mathbf{r}^i \times (\mathbf{T}_{-\delta^i}^T \cdot \mathbf{F}_{tire}^i))_z. \tag{5.15}$$

With the knowledge of the total force and moment acting on the vehicle, the time derivatives of the state variables can be calculated as

$$\dot{v}_x = \frac{F_{total,x}}{m} + \dot{\psi} \cdot v_y, \tag{5.16}$$

$$\dot{v}_y = \frac{F_{total,y}}{m} - \dot{\psi} \cdot v_x, \tag{5.17}$$

$$\ddot{\psi} = \frac{M_z}{I_z}. \tag{5.18}$$

In order to use the described state transition function in a discrete time domain, an integration step is needed to obtain the state variables at the next time step. This can be done with a forward Euler method. The integrations step can be written as

$$\mathbf{x}[k+1] = \mathbf{x}[k] + f(\mathbf{x}[k], \mathbf{u}[k]) \cdot dt. \tag{5.19}$$

Implementation and results

The model based speed estimator was also implemented in MATLAB Simulink envionment. The function that realizes the described calculation can be found at `Conventional_`

`estimators/Model_based_speed_estimator.m`. The estimator was tested on the same test dataset and the results of the evaluation can be seen in table 5.3.

The model parameters that were used for the estimation can be seen in table 5.2.

Parameter	**Value**
Vehicle mass (m)	3020 $[kg]$
Yaw moment of inertia (I_z)	7145 $[kgm^2]$
Front longitudinal tire stiffness (c_{lon}^F)	180000 $[N]$
Rear longitudinal tire stiffness (c_{lon}^R)	200000 $[N]$
Front lateral tire stiffness (c_{lat}^F)	140000 $[N]$
Rear lateral tire stiffness (c_{lat}^R)	150000 $[N]$
Effective wheel radius (R)	0.3615 [m]
Positional vector to front left wheel (r^{FL})	$[1.675, 0.831, 0]^T$ [m]
Positional vector to front right wheel (r^{FR})	$[1.675, -0.831, 0]^T$ [m]
Positional vector to rear left wheel (r^{RL})	$[-1.540, 0.842, 0]^T$ [m]
Positional vector to rear right wheel (r^{RR})	$[-1.540, -0.842, 0]^T$ [m]
Small constant for slip calculation (ϵ)	1

Table 5.2: Model parameters used for the model based speed estimator

RMSE $[\frac{km}{h}]$	**MAE** $[\frac{km}{h}]$	**AMaxE** $[\frac{km}{h}]$	**PWT 0.3 [%]**	**PWT 0.4 [%]**
0.30949	0.19039	2.18090	74.22867	76.07354

Table 5.3: Performance metrics of the model based speed estimator on the test dataset

The model based speed estimator shows a significant improvement in performance compared to the best wheel based speed estimator. This indicates that the developed vehicle model is able to capture the dynamics of the vehicle more accurately, leading to better speed estimation. However the performance still falls behind the AI based direct estimators from the previous chapter.

The major downside of this estimation method is that in low speed scenarios (below 5 km/h) the estimated values tend to be less accurate and unable to estimate correctly in near zero situations. This is due to the fact that at low speeds the tire forces are small and the linear tire model used in the estimator does not accurately represent the tire behavior. This can be observed in figure 5.2, where the estimator shows larger errors during low speeds. However, during higher speed driving scenarios (above 5 km/h) the estimator performs significantly better even being able to estimate much better in situations of steering. It is important to note that this estimation method relies on the linear tire model, which is not accurate under high slip conditions.

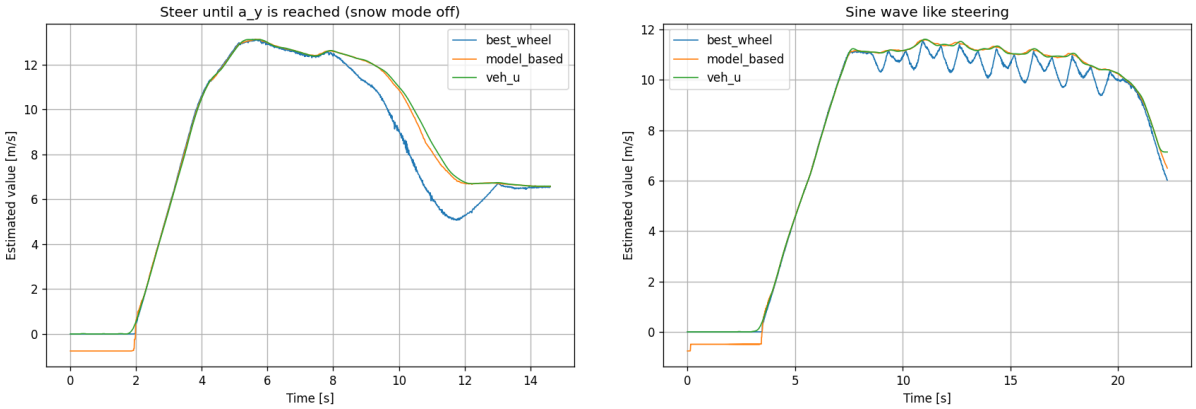


Figure 5.2: Steer until a_y is reached (left) and sine wave like steering (right)

5.3 Extended Kalman Filter based estimation

5.3.1 EKF theory

The Extended Kalman Filter (EKF) is an extension of the original Kalman Filter (KF) that is designed to handle non-linear systems. While the standard KF assumes that both the state transition and measurement functions are linear, the EKF allows for non-linear functions by linearizing them around the current estimate. However the non-linear state transition and measurement functions need to be differentiable. The two functions can be expressed as:

$$\mathbf{x}[k+1] = f(\mathbf{x}[k], \mathbf{u}[k]) + w[k], \tag{5.20}$$

$$\mathbf{z}[k] = h(\mathbf{x}[k]) + \mathbf{v}[k], \tag{5.21}$$

where $\mathbf{x}[k+1]$ is the state vector at time step $k+1$, $\mathbf{u}[k-1]$ is the input vector at time step $k-1$, $\mathbf{z}[k]$ is the measurement vector at time step k, f is the non-linear state transition function, h is the non-linear measurement function, $\mathbf{w}[k-1]$ is the process noise and $v[k]$ is the measurement noise. Both noise terms are assumed to be additive zero-mean Gaussian with known covariance matrices Q and R respectively.

The EKF algorithm consists of two main steps: prediction and update. In the prediction step, the priori state estimation ($\hat{x}_{k|k-1}$) and the priori error covariance ($P_{k|k-1}$) are calculated using the non-linear state transition function:

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}), \tag{5.22}$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_{k-1}\mathbf{P}_{k-1|k-1}\mathbf{F}_{k-1}^T + \mathbf{Q}, \tag{5.23}$$

where $\mathbf{F}_{k-1}$ is the Jacobian of the state transition function evaluated at the previous state

estimate:

$$\mathbf{F}_{k-1} = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{\mathbf{x}=\hat{\mathbf{x}}_{k-1|k-1},\mathbf{u}=\mathbf{u}_{k-1}}. \tag{5.24}$$

In the update step, the Kalman gain ($\mathbf{K}_k$), the posteriori state estimate ($\hat{\mathbf{x}}_{k|k}$) and the updated error covariance ($\mathbf{P}_{k|k}$) are calculated using the non-linear measurement function:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}_k^T(\mathbf{H}_k\mathbf{P}_{k|k-1}\mathbf{H}_k^T + \mathbf{R})^{-1}, \tag{5.25}$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k(\mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})), \tag{5.26}$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k\mathbf{H}_k)\mathbf{P}_{k|k-1}, \tag{5.27}$$

where $\mathbf{H}_k$ is the Jacobian of the measurement function evaluated at the predicted state estimate:

$$\mathbf{H}_k = \left.\frac{\partial h}{\partial \mathbf{x}}\right|_{\mathbf{x}=\hat{\mathbf{x}}_{k|k-1}}. \tag{5.28}$$

5.3.2 EKF implementation

EKF with best wheel based speed estimation

As mentioned in the previous section, there are two main components that need to be defined in order to implement the EKF based estimator: the state transition function and the measurement function. The state transition function will be the previously developed model in section 5.2, that calculates the vehicle speed derivatives based on the vehicle model and the sensor measurements. The measurement function will be first the best wheel based speed estimator presented in section 5.1, that provides an estimate of the vehicle speed based on the wheel speeds. Later the best performing AI based direct vehicle speed estimator from section 4.5 will be used as the measurement function.

To be able to use the best features of the model based estimator and the best wheel based estimator, the EKF will be set in a way that in low speed scenarion (below 5 km/h) the covariance of the process noise ($Q(v)$) will be set to lower and the covariance of the observation noise to higher values, indicating to trust the measurements more. After passing that limit the covariance value $R(v)$ will be set to lower and $Q(v)$ to higher values in order to trust the model based estimator more. This way the EKF can provide a more accurate estimate in both low speed and high speed scenarios.

The implemented EKF based estimator achieved a significant improvement in performance compared to both the previously presented best wheel based and model based speed estimators. The results of the evaluation can be seen in table 5.4.

RMSE $[\frac{m}{s}]$	MAE $[\frac{m}{s}]$	AMaxE $[\frac{m}{s}]$	PWT 0.3 [%]	PWT 0.4 [%]
0.16694	0.07886	3.35581	94.23076	96.01073

Table 5.4: Performance metrics of the EKF based speed estimator with best wheel based measurement function on the test dataset

In order to see the performance improvement of the EKF based estimator compared to the individual estimators, figure 5.3 shows an example of the estimation results during a cornering maneuver. It can be seen that the EKF based estimator is able to provide a more accurate estimate by combining the strengths of both the model based and the best wheel based estimators.

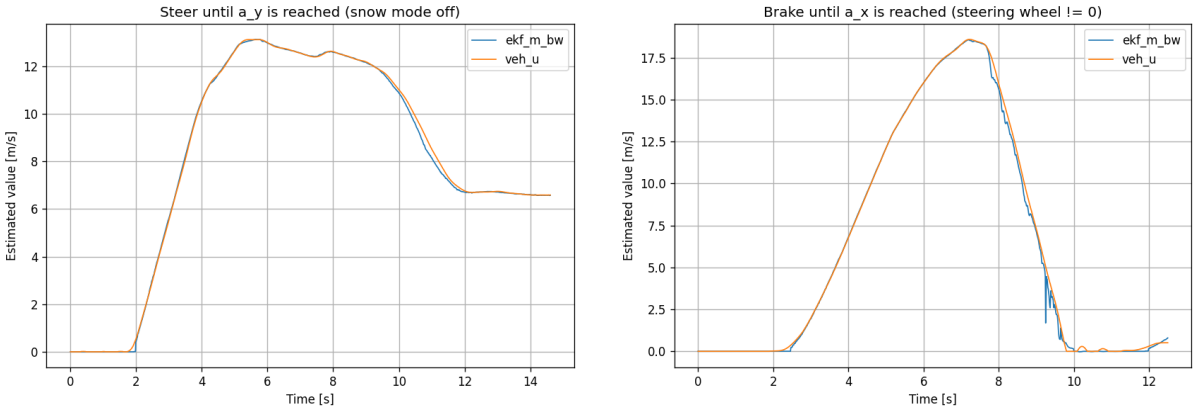


Figure 5.3: Steer until a_y is reached (left) and brake until a_x is reached (right)

It can be seen on the second figure that during braking maneuvers the model based state update transition function tends to be less accurate due to the linear tire model.

EKF with AI based speed estimation

In this section the best performing AI based direct vehicle speed estimator from section 4.5 will be used as the measurement function in the EKF based estimator. The state transition function will remain the same as in the previous section, which is the model based speed estimator presented in section 5.2. The goal is to see if the combination fo the two methods can provide a better performance compared to the individual estimators.

The covariance tuning strategy will be the following: in low speed scenarios (below 5 km/h) the covariance of the process noise ($Q(v)$) will be set to higher value, indicating to trust the measurements more, as the model based estimator tends to be less accurate as it was seen before. After passing that limit the diagonal elements of the covariance matrix $Q(v)$ will be set to the same value as the covariance of the observation noise (R), indicating to trust both the model based and AI based estimators equally.

The values of the covariance matrices used in the EKF implementation can be seen in

the following equations:

$$Q = \begin{bmatrix} 100 & 0 & 0 \\ 0 & 0.01 & 0 \\ 0 & 0 & 0.01 \end{bmatrix} \text{ if } v_u < 5[\frac{km}{h}], \quad Q = \begin{bmatrix} 0.01 & 0 & 0 \\ 0 & 0.01 & 0 \\ 0 & 0 & 0.01 \end{bmatrix} \text{ if } v_u \geq 5[\frac{km}{h}], \tag{5.29}$$

$$R = 0.01. \tag{5.30}$$

Prioir to the results, it has to be noted that the TCN based direct AI estimator was performing better in each individual validation test compared to the model based estimator, which serves as the state transition function in the EKF. Therefore it is expected that the EKF based estimator will not be able to outperform the AI based estimator, only if the estimator would omit the model based state update function and rely solely on the AI based measurement function. However, the advantage of this setup is that it could provide more robust estimates in scenarios where the AI model might fail, such as in out-of-distribution situations, that were not present in the training data.

The performance indices of the implemented EKF based estimator with AI based measurement can be seen in table 5.5. On figure 5.4 two examples of the estimation can be seen, where on the left the estimator is the most accurate and on the right where the estimator shows larger errors due to high lateral slip during cornering.

RMSE $[\frac{m}{s}]$	**MAE** $[\frac{m}{s}]$	**AMaxE** $[\frac{m}{s}]$	**PWT 0.3 [%]**	**PWT 0.4 [%]**
0.0843	0.0433	1.8623	98.4661	98.9649

Table 5.5: Performance metrics of the EKF based speed estimator with AI based measurement function on the test dataset

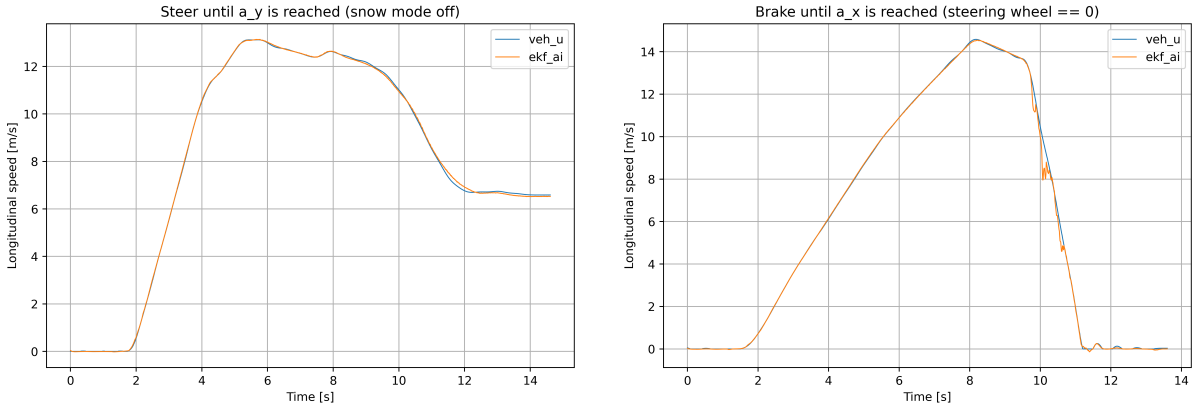


Figure 5.4: Low longitudinal and lateral slip in straight line driving (left) and high lateral slip during cornering (right)

It can be seen that the results reflect the expactations mentioned before. The estimator did not outperform the individual AI based direct estimator, however it is more likely to

provide robust estimates in driving scenarions and dynamic ranges that were not present during the training of the AI model.

Chapter 6

Real Time Impelementation

In this chapter the real time implementation of the trained networks will be described. The main focus will be on integrating the model into an environment where it can process the incoming data in real time and provide estimations with as little latency as possible.

6.1 Development Environment

Before describing the development environment, the meaning behind real time implementation should be clarified. In this context, real time means that the model is able to process incoming data and provide estimations within a fixed time frame, which is determined by the system. Within every defined time step, the computational unit (eg. microcontroller) receives new input data at the beginning of the time step, processes it using the implemented model, and outputs the estimation before the next time step begins. This setup can be seen on figure 6.1

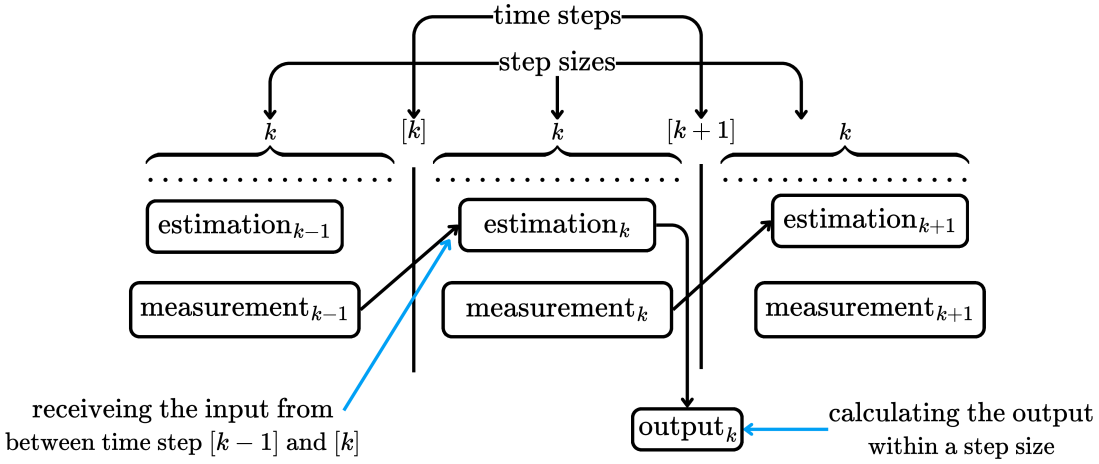


Figure 6.1: Real time implementation concept

A possible development environment for implementing an arbitrary control or estimation algorithm for vehicle application can be a toolchain offered by dSpace. This toolchain

allows the user to create models in MATLAB Simulink, generate C code from these models and utilize the generated code on the hardware developed by dSace, where it can be tested in real time. The hardware can be connected to a real vehicle via the CAN bus, or to a simulator for cost efficiency reasons. In this thesis the latter application was chosen, using the BeamNG.drive simulator. The main steps of the real time implementations are the following:

1. Prepare the trained models in MATLAB using the Deep Learning Toolbox (DLT).
2. Integrate the trained models into MATLAB Simulink using DLT.
3. Generating C code from the Simulink model using dSpace's ConfigurationDesk software.
4. Deploying the generated code on dSpace's MicroAutoBox hardware using ControlDesk software.
5. Connecting the MicroAutoBox to computer that runs BeamNG.drive via CAN bus.
6. Testing the real time performance of the models with different step sizes.

The toolchain of the real time implementation is illustrated in Figure 6.2.

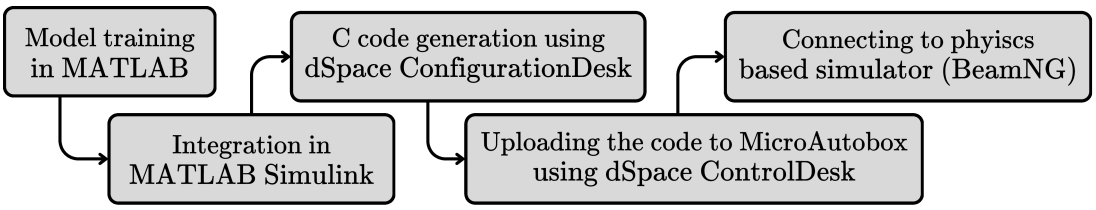


Figure 6.2: Toolchain of the real time implementation

The main question to answer is what is the smallest step size that the MicroAutoBox requires to be able to run the model. In other words, how much time does the MicroAutoBox need to process the inserted model and provide and estimation. This is crucial question since the smaller the step size, the more accurate and responsive the estimation will be, which can result better overall performance in a control application.

6.2 Results

The chosen models for real time implementation were the best performing LSTM, GRU and the TCN architectures described in Chapter 4. Before starting the tests, a short calculation was made to estimate the FLOPS (floating point operations per second) required by each model. This gives a rough idea about the computational requirements of the models, which can be compared to the capabilities of the MicroAutoBox hardware. The calculated FLOPS are the following:

- GRU:

$$\text{seq len} \times \text{hidden size}^2 \times \text{layers} \times \text{gate factor} = 90 \times 208^2 \times 3 \times 6 \simeq 70.088 \text{ MFLOPS} \quad (6.1)$$

- LSTM:

$$\text{seq len} \times \text{hidden size}^2 \times \text{layers} \times \text{gate factor} = 50 \times 192^2 \times 2 \times 8 \simeq 29.491 \text{ MFLOPS} \quad (6.2)$$

- TCN:

$$\text{seq len} \times \text{channels} \times \text{kernel} \times \text{layers} \times \text{conv per block} = 50 \times 64 \times 4 \times 4 \times 2 \simeq 0.012 \text{ MFLOPS} \quad (6.3)$$

The results indicate that the TCN model has significantly lower computational cost compared to the recurrenet models which makes it even more desirable for real time applications.

In table 6.1 the highest task turnaround times that were read out on the MicroAutoBox are indicated for the specific models and an adivsable minimal time step for each model.

Model	**Max. turnaround time [ms]**	**Advisable min. step size [ms]**
LSTM	68.4	100
GRU	124.5	200
TCN	25.8	50

Table 6.1: Real time implementation results on dSpace MicroAutoBox

After measuring these step sizes, it is clear that the TCN model is the most suitable for real time application due to its low computational cost and high accuracy. The main difference between the RNN based models is the sequence lenght which greatly affects computational costs. The LSTM model has a sequence length of 50 while the GRU model has a length of 90, which results in almost double the computational cost for the GRU model despite having only one layer more than the LSTM model.

Chapter 7

Summary

This thesis focused on the development and evaluation of various estimators for vehicle speed estimation using conventional and artificial intelligence based methods. The main objective was to discover the usage of AI algorithms in vehicle speed estimation and compare their performance to already well developed methods.

The thesis began with discovering the vehicle as a system of sensors that serve for state estimation purposes. From the available sensor signals the relevant ones for vehicle speed estimation where selected and preprocessed for further usage. Following this, a direct estimation approach was presented where different sequential AI architectures such as RNN including LSTM and GRU, TCN and Transformer were implemented and evaluated. The hyperparameter optimization of these models was performed through grid search, and the best performing models were identified based on their performance on a validation dataset. The results indicated that the TCN architecture outperformed both RNN and Transformer models in terms of RMSE, MAE, and PWT metrics.

In the subsequent chapter, conventional vehicle speed estimation methods were presented to compare the AI based appraches with. These methods included the best wheel based estimator, linear tire model based estimator and two Extended Kalman Filter based estimators. The performance of these conventional methods was evaluated on the same dataset as the AI models, allowing for a direct comparison of their effectiveness in vehicle speed estimation, where it was found that the best performing AI based direct estimation morel outperformed all the conventional methods.

Finally, the thesis exploited the best AI based direct estimation model in a real-time implementation scenario. The model was integrated into a real-time system, and its performance was assessed under real-world conditions. The results demonstrated that the AI based estimator could operate effectively in real-time but required careful consideration of computational resources and latency.

In conclusion, this thesis demonstrated the potential of AI algorithms in vehicle speed estimation, showing that they can outperform conventional methods but also come with challanges related to real-time implementation.

Bibliography

[1] *How to calculate wheel and vehicle speed from engine speed.* URL: https://x-engineer.org/calculate-wheel-vehicle-speed-engine-speed/ (visited on 05/05/2025).

[2] The Editors of Encyclopaedia Britannica. *speedometer.* URL: https://www.britannica.com/technology/speedometer (visited on 05/07/2025).

[3] Chris Woodford. *Speedometers.* URL: https://www.explainthatstuff.com/how-speedometer-works.html (visited on 05/05/2025).

[4] Otto Schulze. "Improvements in Speed Indicators". Pat. GB000190322622A (Germany). May 20, 1903.

[5] Joseph W Jones. "Speedometer". Pat. US765841A (United States). July 26, 1904.

[6] *Wheel Speed Sensors – More than just ABS.* URL: https://premierautotrade.com.au/news/wheel-speed-sensors%E2%80%93more-than-just-abs.php (visited on 05/07/2025).

[7] *What is GNSS.* URL: https://www.euspa.europa.eu/eu-space-programme/galileo/what-gnss (visited on 05/07/2025).

[8] *GPS satellites - trilateration.* URL: https://openclipart.org/detail/191659/gps-satellites-trilateration (visited on 05/08/2025).

[9] *Math frames.* URL: https://www.vectornav.com/resources/inertial-navigation-primer/math-fundamentals/math-refframes (visited on 05/08/2025).

[10] Chul Ki Song, Michael Uchanski, and J. Hedrick. "Vehicle Speed Estimation Using Accelerometer and Wheel Speed Measurements". In: July 2002. DOI: 10.4271/2002-01-2229.

[11] Lars Nielsen Uwe Kiencke. *Automotive Control Systems: For Engine, Driveline, and Vehicle.* Springer Berlin, Heidelberg, 2005.

[12] Giulio Reina, Matilde Paiano, and Jose-Luis Blanco-Claraco. "Vehicle parameter estimation using a model-based estimator". In: *Mechanical Systems and Signal Processing* 87 (2017). Signal Processing and Control challenges for Smart Vehicles, pp. 227–241. ISSN: 0888-3270. DOI: https://doi.org/10.1016/j.ymssp.2016.06.038. URL: https://www.sciencedirect.com/science/article/pii/S0888327016302205.

[13] Li Ji et al. "Evaluation of the performance of GNSS-based velocity estimation algorithms". In: *Satellite Navigation* (2022).

[14] John Chrosniak, Jingyun Ning, and Madhur Behl. "Deep Dynamics: Vehicle Dynamics Modeling With a Physics-Constrained Neural Network for Autonomous Racing". In: *IEEE Robotics and Automation Letters* 9.6 (June 2024), pp. 5292–5297. ISSN: 2377-3774. DOI: 10.1109/lra.2024.3388847. URL: http://dx.doi.org/10.1109/LRA.2024.3388847.

[15] Taekyung Kim, Hojin Lee, and Wonsuk Lee. *Physics Embedded Neural Network Vehicle Model and Applications in Risk-Aware Autonomous Driving Using Latent Features*. 2022. arXiv: 2207.07920 [cs.RO]. URL: https://arxiv.org/abs/2207.07920.

[16] Zhuangwei Shi. "Incorporating Transformer and LSTM to Kalman Filter with EM algorithm for state estimation". In: *CoRR* abs/2105.00250 (2021). arXiv: 2105.00250. URL: https://arxiv.org/abs/2105.00250.

[17] N. Yadaiah and G. Sowmya. "Neural Network Based State Estimation of Dynamical Systems". In: *The 2006 IEEE International Joint Conference on Neural Network Proceedings*. 2006, pp. 1042–1049. DOI: 10.1109/IJCNN.2006.246803.

[18] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling". In: *CoRR* abs/1803.01271 (2018). arXiv: 1803.01271. URL: http://arxiv.org/abs/1803.01271.

[19] Ashish Vaswani et al. "Attention is All You Need". In: 2017. URL: https://arxiv.org/pdf/1706.03762.pdf.

[20] Rajesh Rajamani. *Vehicle Dynamics and Control*. Springer, 2012.

List of Figures

List of Tables

Appendix A

Appendix Title

At the end of the thesis, appendices can be used to include code snippets, catalog pages, large images, etc. This is referenced as 1. appendix in the text. It is recommended to use an online version control system (e.g., GitHub) to publicly upload the project and provide the link in the thesis.

Appendix B

Appendix Title

Model ID	Type	Input ID	Hidden size	Sequence length	Layers
153.1	RNN	2	50	64	1
153.2	RNN	1	50	64	1
154.1	RNN	2	100	64	1
154.2	RNN	1	100	64	1
155.1	RNN	2	150	64	1
155.2	RNN	1	150	64	1
156.1	RNN	2	200	64	1
156.2	RNN	1	200	64	1
157.1	RNN	2	50	96	1
157.2	RNN	1	50	96	1
158.1	RNN	2	100	96	1
158.2	RNN	1	100	96	1
159.1	RNN	2	150	96	1
159.2	RNN	1	150	96	1
160.1	RNN	2	200	96	1
160.2	RNN	1	200	96	1
161.1	RNN	2	50	128	1
161.2	RNN	1	50	128	1
162.1	RNN	2	100	128	1
162.2	RNN	1	100	128	1
163.1	RNN	2	150	128	1
163.2	RNN	1	150	128	1
164.1	RNN	2	200	128	1
164.2	RNN	1	200	128	1
165.1	RNN	2	50	192	1
165.2	RNN	1	50	192	1
166.1	RNN	2	100	192	1
166.2	RNN	1	100	192	1
167.1	RNN	2	150	192	1
167.2	RNN	1	150	192	1
168.1	RNN	2	200	192	1
168.2	RNN	1	200	192	1
169.1	RNN	2	50	64	2
169.2	RNN	1	50	64	2
170.1	RNN	2	100	64	2
170.2	RNN	1	100	64	2
171.1	RNN	2	150	64	2
171.2	RNN	1	150	64	2
172.1	RNN	2	200	64	2
172.2	RNN	1	200	64	2
173.1	RNN	2	50	96	2
173.2	RNN	1	50	96	2
174.1	RNN	2	100	96	2
174.2	RNN	1	100	96	2
175.1	RNN	2	150	96	2
175.2	RNN	1	150	96	2
176.1	RNN	2	200	96	2
176.2	RNN	1	200	96	2
177.1	RNN	2	50	128	2
177.2	RNN	1	50	128	2
178.1	RNN	2	100	128	2